



Active Suspension (Quarter-Car) Control Project

Students: Abdelrahman Mohammed (202200281)
Omar Elmetwalli (202200057)
Rana Sabry (202200791)
Roaa Gamaleldin (202200350)

Instructor: Eng. Mariam Hesham
Supervisor: Dr. Mohannad Draz

January 1, 2026

Abstract

This report models a quarter-car active suspension with sprung mass m_1 (vehicle body) and unsprung mass m_2 (wheel assembly). Road displacement W excites the wheel, and a control force u acts between the two masses to reduce suspension deflection $y_1 = x_1 - x_2$. We derive the equations of motion, verify the provided transfer matrix (Part 3), select a parameter set, compute step-response metrics for $G_1 = \frac{x_1 - x_2}{u}$ and $G_2 = \frac{x_1 - x_2}{W}$, build a state-space model with output y_1 , analyze stability/controllability/observability, and design a state-feedback controller meeting the disturbance requirements. Finally, we describe the Simulink implementation and verification plots for $y_1(t)$ and $u(t)$.

Contents

1	Introduction	4
2	Problem Description	4
3	Part (2): Modeling Equations	4
4	Part (3): Verify the Transfer Matrix (Cramer's Rule)	5
5	Part (4): Parameters and Step Responses of G_1, G_2	6
5.1	Assumed parameter set (typical quarter-car values)	6
5.2	Transfer functions for $y = x_1 - x_2$	6
5.3	Results	7
5.3.1	MATLAB Results: Part 4	7
6	Part (5): State-Space Model with Output $y_1 = x_1 - x_3$	8
7	Part (6): Open-Loop Stability	8
8	Part (7): Controllability and Observability	8
9	Part (8): State-Feedback Controller Design	8
9.1	Pole selection (with calculations)	8
9.2	Control law	10
9.3	Chosen poles and computed gains	10
9.4	Closed-loop verification	10
10	Part (9): Simulink Verification (Wiring + Block Settings)	11
10.1	Workspace variables	11
10.2	Wiring summary	11
11	Passive suspension with real road generator	12
11.1	Road input definition for suspension simulation	12
11.2	ISO 8608 road roughness model in the spatial frequency domain	12
11.3	Spectral synthesis of the spatial road profile	13
11.3.1	Derivation of the amplitude formula	13
11.4	Conversion to the time-domain road input $W(t)$	14
11.5	Parameter selection and realism justification	14
11.6	MATLAB Implementation and Simulation Results	15
11.7	MATLAB Implementation with Ramp and Bump Road Inputs	16
12	LQR State-Feedback Design	18
12.1	Objective and requirements (Project)	18
12.2	LQR formulation and control law	18
12.3	How the LQR weights Q and R were selected (and tuned)	18
12.4	Road/Terrain Disturbance Modeling as a General Input $W(t)$	19
12.4.1	What W means physically: road profile as height vs. distance	19
12.4.2	How $W(t)$ enters the suspension dynamics	20
12.4.3	Numerical simulation used in this project: ZOH discretization	20

12.4.4	Deterministic terrain components (waves and bumps)	20
12.4.5	Stochastic roughness component (math-correct noise shaping) . .	21
12.4.6	Final discrete terrain sequence used by the plant	22
12.5	Simulink implementation (Task 9 repeated with LQR)	22
12.6	Results and discussion	23
References		28
A Appendix A: MATLAB Scripts		30
A.1	MainSimulation.m	30
A.2	Passive suspension ISO 8608.m	35
A.3	Roadsimulation.m	40
A.4	rampthenbump.m	41
A.5	Newiteration.m	47
A.6	newii.m	52
A.7	lqr_pcode.m	52
A.8	animation_gui	54
A.9	LQR python comfort analysis animation (animation.py)	59
B LQR Simulink Model attributes		64
B.1	Full model	64
B.2	Unit-step response model attributes	66

1 Introduction

The quarter-car model represents:

- m_1 : sprung mass (vehicle body/quarter chassis mass),
- m_2 : unsprung mass (wheel + axle assembly),
- k_1, b_1 : suspension spring and damper between m_1 and m_2 ,
- k_2, b_2 : tire stiffness and damping between m_2 and the road,
- W : road disturbance displacement,
- u : active control force applied between the masses to reduce deflection $x_1 - x_2$ and improve ride comfort.

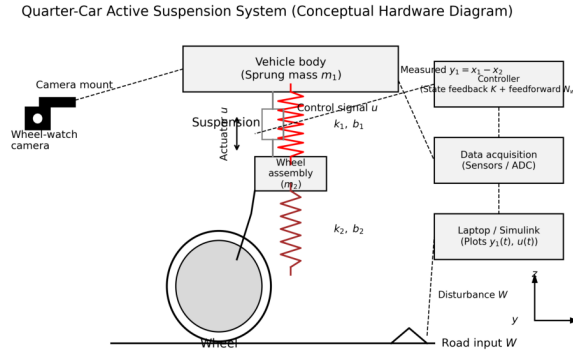


Figure 1: Quarter-car active suspension model

2 Problem Description

We model the active suspension and design feedback control so that under a 0.1 m step road disturbance the suspension deflection $y_1 = x_1 - x_3$ stays within ± 0.1 m and oscillations vanish within 5 s. The design is verified in Simulink with plots of $y_1(t)$ and the control input $u(t)$.

3 Part (2): Modeling Equations

Take upward displacement positive for x_1, x_2, W . The actuator applies equal/opposite forces: $(+u)$ on m_1 , $(-u)$ on m_2 .

Suspension (between m_1 and m_2):

- Spring force: $k_1(x_1 - x_2)$
- Damper force: $b_1(\dot{x}_1 - \dot{x}_2)$

Tire (between m_2 and road W):

- Spring force: $k_2(x_2 - W)$

- Damper force: $b_2(\dot{x}_2 - \dot{W})$

Equations of motion:

$$m_1\ddot{x}_1 = -b_1(\dot{x}_1 - \dot{x}_2) - k_1(x_1 - x_2) + u, \quad (1)$$

$$m_2\ddot{x}_2 = b_1(\dot{x}_1 - \dot{x}_2) + k_1(x_1 - x_2) - b_2(\dot{x}_2 - \dot{W}) - k_2(x_2 - W) - u. \quad (2)$$

4 Part (3): Verify the Transfer Matrix (Cramer's Rule)

Taking Laplace transforms of (1)–(2) (zero initial conditions) gives:

$$(M_1s^2 + b_1s + k_1)X_1 - (b_1s + k_1)X_2 = U, \quad (3)$$

$$-(b_1s + k_1)X_1 + (M_2s^2 + (b_1 + b_2)s + (k_1 + k_2))X_2 = -U + (b_2s + k_2)W. \quad (4)$$

Write (3)–(4) in matrix form:

$$A(s) \begin{bmatrix} X_1 \\ X_2 \end{bmatrix} = \begin{bmatrix} U \\ -U + (b_2s + k_2)W \end{bmatrix}, \quad A(s) = \begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{bmatrix} \quad (5)$$

where

$$a_{11} = M_1s^2 + b_1s + k_1, \quad a_{12} = a_{21} = -(b_1s + k_1), \quad a_{22} = M_2s^2 + (b_1 + b_2)s + (k_1 + k_2).$$

Note (what we did) — Cramer's Rule Steps

In this project, Part (3) was solved using *Cramer's rule*. From

$$A(s) \begin{bmatrix} X_1 \\ X_2 \end{bmatrix} = \begin{bmatrix} U \\ -U + (b_2s + k_2)W \end{bmatrix}, \quad A(s) = \begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{bmatrix},$$

Cramer's rule for a 2×2 system gives:

$$X_1(s) = \frac{\Delta_1(s)}{\Delta(s)}, \quad X_2(s) = \frac{\Delta_2(s)}{\Delta(s)}.$$

Step 1: Compute the main determinant $\Delta(s)$

$$\Delta(s) = \det(A(s)) = \begin{vmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{vmatrix} = a_{11}a_{22} - a_{12}a_{21}.$$

Substituting

$$a_{11} = M_1s^2 + b_1s + k_1, \quad a_{12} = a_{21} = -(b_1s + k_1), \quad a_{22} = M_2s^2 + (b_1 + b_2)s + (k_1 + k_2),$$

we get:

$$\Delta(s) = (M_1s^2 + b_1s + k_1)(M_2s^2 + (b_1 + b_2)s + (k_1 + k_2)) - (b_1s + k_1)^2.$$

Step 2: Compute $\Delta_1(s)$ (replace column 1 by the RHS vector)

$$\Delta_1(s) = \begin{vmatrix} U & a_{12} \\ -U + (b_2s + k_2)W & a_{22} \end{vmatrix} = U a_{22} - a_{12} \left(-U + (b_2s + k_2)W \right).$$

So,

$$X_1(s) = \frac{\Delta_1(s)}{\Delta(s)}.$$

Step 3: Compute $\Delta_2(s)$ (replace column 2 by the RHS vector)

$$\Delta_2(s) = \begin{vmatrix} a_{11} & U \\ a_{21} & -U + (b_2s + k_2)W \end{vmatrix} = a_{11} \left(-U + (b_2s + k_2)W \right) - U a_{21}.$$

So,

$$X_2(s) = \frac{\Delta_2(s)}{\Delta(s)}.$$

Final step After finding $X_1(s)$ and $X_2(s)$, we compute the required output:

$$Y(s) = X_1(s) - X_2(s),$$

and this leads to the same input–output transfer matrix stated in the project.

5 Part (4): Parameters and Step Responses of G_1, G_2

5.1 Assumed parameter set (typical quarter-car values)

We used the following parameters (commonly used in quarter-car examples):

Parameter	Value	Unit
m_1	290	kg
m_2	59	kg
k_1	16000	N/m
k_2	190000	N/m
b_1	1000	N·s/m
b_2	0	N·s/m

Table 1: Quarter-car parameters used in simulation.

Remark: Setting $b_2 = 0$ avoids introducing the additional input $\dot{W}$ in the state-space model.

5.2 Transfer functions for $y = x_1 - x_2$

From Part (3):

$$G_1(s) = \frac{X_1 - X_2}{U} = \frac{(m_1 + m_2)s^2 + b_2s + k_2}{\Delta(s)}, \quad (6)$$

$$G_2(s) = \frac{X_1 - X_2}{W} = \frac{-m_1s^2(b_2s + k_2)}{\Delta(s)}. \quad (7)$$

5.3 Results

The project requires a road step of $W = 0.1$ m.

Table 2: Step-response characteristics of G_1 and G_2

Case	Input	Key results	Notes
G_1	Unit step in u	$T_s = 2.38$ s $M_p = 47.39\%$ $\omega_d = 58.03$ rad/s	Scale output with step amplitude
G_2	0.1 m step in W	$\max y = 0.137$ m $T_s = 5.00$ s $\omega_d = 58.03$ rad/s	Poor ride comfort

Table 3: Step-response metrics used in the report.

5.3.1 MATLAB Results: Part 4

```

1 ===== G1 RESULTS =====
2 Settling Time = 2.384 s
3 Maximum Overshoot = 47.39 %
4 Damped Frequency wd = 58.030 rad/s
5
6 ===== G2 RESULTS =====
7 Peak Deflection = 0.1372 m (137.2 % of 0.1 m)
8 Settling Time = 5.000 s
9 Damped Frequency wd = 58.030 rad/s

```

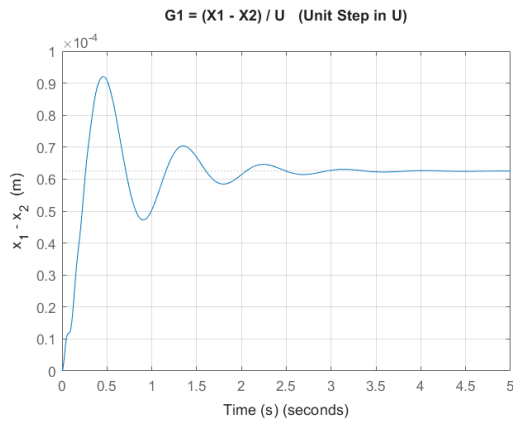


Figure 2: Step response of $G_1 = \frac{X_1 - X_2}{U}$.

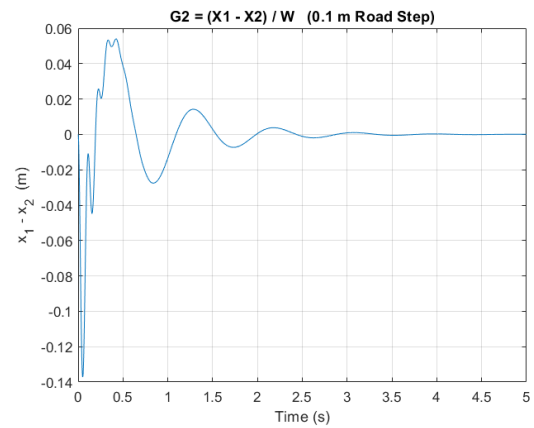


Figure 3: Step response of $G_2 = \frac{X_1 - X_2}{W}$ for $W = 0.1$ m.

6 Part (5): State-Space Model with Output $y_1 = x_1 - x_3$

Choose the state vector:

$$x = [x_1 \quad \dot{x}_1 \quad x_2 \quad \dot{x}_2]^\top, \quad y_1 = [1 \quad 0 \quad -1 \quad 0] x. \quad (8)$$

For $b_2 = 0$:

$$\dot{x} = Ax + B_u u + B_w W, \quad y_1 = Cx, \quad (9)$$

where

$$A = \begin{bmatrix} 0 & 1 & 0 & 0 \\ -\frac{k_1}{m_1} & -\frac{b_1}{m_1} & \frac{k_1}{m_1} & \frac{b_1}{m_1} \\ 0 & 0 & 0 & 1 \\ \frac{k_1}{m_2} & \frac{b_1}{m_2} & -\frac{k_1+k_2}{m_2} & -\frac{b_1+b_2}{m_2} \end{bmatrix}, \quad B_u = \begin{bmatrix} 0 \\ \frac{1}{m_1} \\ 0 \\ -\frac{1}{m_2} \end{bmatrix}, \quad B_w = \begin{bmatrix} 0 \\ 0 \\ 0 \\ \frac{k_2}{m_2} \end{bmatrix}, \quad C = [1 \quad 0 \quad -1 \quad 0]. \quad (10)$$

Note: If $b_2 \neq 0$, an additional input proportional to $\dot{W}$ appears because the tire damper uses $(\dot{x}_2 - \dot{W})$.

7 Part (6): Open-Loop Stability

The eigenvalues of A (equal to the transfer-function poles) for the chosen parameters are:

$$\lambda(A) = -8.7169 \pm j58.0297, \quad -1.4818 \pm j7.0287. \quad (11)$$

All real parts are negative, so the open-loop system is asymptotically stable.

8 Part (7): Controllability and Observability

Using input u and output $y_1 = x_1 - x_3$:

- The controllability matrix $\mathcal{C} = [B_u, AB_u, A^2B_u, A^3B_u]$ has rank 4 $\Rightarrow$ the system is **fully controllable**.
- The observability matrix $\mathcal{O} = [C^\top \quad (CA)^\top \quad (CA^2)^\top \quad (CA^3)^\top]^\top$ has rank 4 $\Rightarrow$ the system is **fully observable** from y_1 .

9 Part (8): State-Feedback Controller Design

Requirement: for a 0.1 m road step, y_1 must stay within ± 0.1 m and oscillations vanish within 5 s.

9.1 Pole selection (with calculations)

For a dominant complex-conjugate pole pair

$$p = -\sigma \pm j\omega_d,$$

the standard 2nd-order relations are:

$$\omega_n = \sqrt{\sigma^2 + \omega_d^2}, \quad \zeta = \frac{\sigma}{\omega_n},$$

$$T_s(2\%) \approx \frac{4}{\zeta \omega_n} = \frac{4}{\sigma}, \quad M_p \approx e^{-\frac{\pi \zeta}{\sqrt{1-\zeta^2}}} \times 100\%.$$

(A) Dominant poles (body mode)

We selected the dominant pair:

$$p_{1,2} = -7.91 \pm j 5.90.$$

So:

$$\sigma = 7.91, \quad \omega_d = 5.90$$

$$\omega_n = \sqrt{7.91^2 + 5.90^2} = \sqrt{62.5681 + 34.81} = 9.868 \text{ rad/s}$$

$$\zeta = \frac{7.91}{9.868} = 0.802$$

At first

$$\sigma = .8,$$

but we choose to be 7.9 enforce it to make it within our safety ranges Settling time:

$$T_s(2\%) \approx \frac{4}{7.91} = 0.506 \text{ s } (< 5 \text{ s requirement})$$

Percent overshoot prediction:

$$M_p \approx e^{-\frac{\pi(0.802)}{\sqrt{1-0.802^2}}} \times 100 = e^{-4.211} \times 100 \approx 1.48\%.$$

This high damping (small M_p) helps keep y_1 within ± 0.1 m for the road step.

(B) Fast poles (wheel mode)

The remaining poles were chosen farther left so they decay faster and do not dominate:

$$p_{3,4} = -20.19 \pm j 45.53.$$

Settling time for this mode:

$$T_s(2\%) \approx \frac{4}{20.19} = 0.198 \text{ s},$$

which is much faster than the dominant mode (0.506 s), so its effect dies out quickly.

Comparison with open-loop dominant poles

The open-loop dominant poles were approximately:

$$p_{ol} = -1.48 \pm j 7.03.$$

Thus:

$$T_{s,ol} \approx \frac{4}{1.48} = 2.70 \text{ s}, \quad \zeta_{ol} = \frac{1.48}{\sqrt{1.48^2 + 7.03^2}} = 0.206,$$

$$M_{p,ol} \approx e^{-\frac{\pi(0.206)}{\sqrt{1-0.206^2}}} \times 100 \approx 51.6\%.$$

So the controller pole choice increases damping significantly (reducing overshoot) and speeds up decay well within 5 seconds.

9.2 Control law

$$u = -Kx + N_w W \quad (12)$$

where $N_w W$ is a feedforward term chosen so that $y_1 \rightarrow 0$ at steady state for constant W .

9.3 Chosen poles and computed gains

Example desired poles:

$$p_{\text{des}} = \{-7.91 \pm j5.90, -20.19 \pm j45.53\}. \quad (13)$$

Computed gains (for the parameters in Table 1):

$$K \approx \begin{bmatrix} 5753 & 2888 & 20634 & -1525 \end{bmatrix}, \quad N_w \approx 26387. \quad (14)$$

9.4 Closed-loop verification

Closed-loop results for a $W = 0.1$ m step:

- Peak deflection: $\max |y_1| \approx 0.096$ m (**within** ± 0.1 m),
- Settling time: ≈ 0.53 s (**within** 5 s),
- Control effort: $u_{\text{max}} \approx 5.51$ kN, $u_{\text{min}} \approx -3.89$ kN.

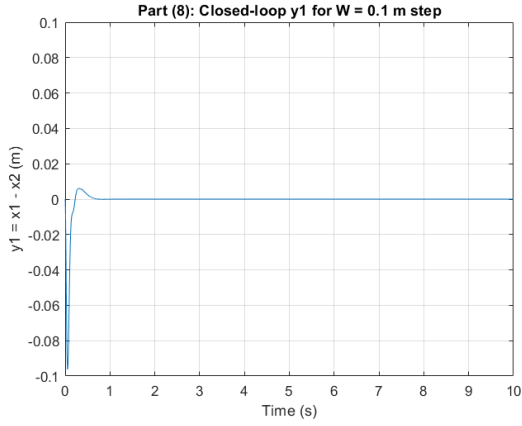


Figure 4: $y_1(t)$

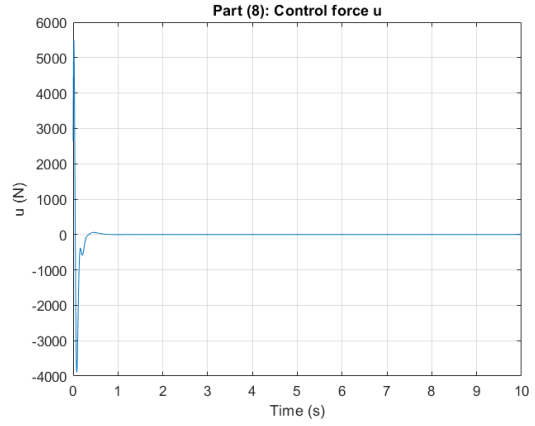


Figure 5: $u(t)$

Figure 6: Closed-loop verification for a road step $W = 0.1$ m.

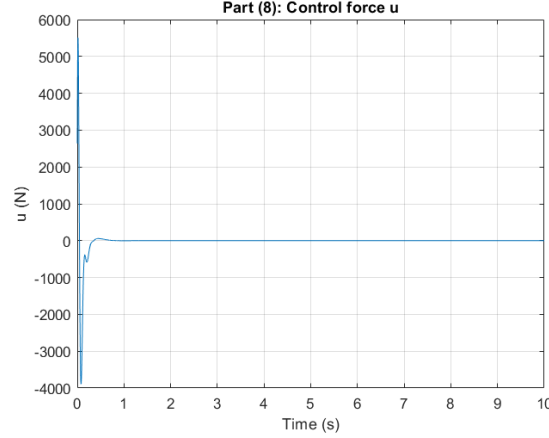


Figure 7: U1 force response

10 Part (9): Simulink Verification (Wiring + Block Settings)

We implemented:

$$u = -Kx + N_w W \quad (15)$$

in Simulink using a **State-Space** block with input vector $[u \ W]^T$ and output y_1 .

10.1 Workspace variables

Before running Simulink, define: **A**, **Bu**, **Bw**, **C_y1**, **D_y1**, **K**, **Nw**. The sizes must be:

- **A**: 4×4 ,
- **B** = $[B_u \ B_w]$: 4×2 ,
- **C_{y1}**: 1×4 ,
- **D_{y1}**: 1×2 (must be $[0 \ 0]$).

10.2 Wiring summary

1. **Step block (road)**: W step (time 0, initial 0, final 0.1).
2. Split W :
 - to **Gain** N_w then to a **Sum** block as (+),
 - to the **State-Space** block as the second input.
3. Enable **Show state port** in the State-Space block to output x .
4. Send x through **Gain** K and feed it to the **Sum** block with a minus sign so the sum computes $-Kx + N_w W$.
5. The sum output is u ; feed it to the plant (first input) and to a scope for $u(t)$.

6. Output y_1 goes to a scope for $y_1(t)$.

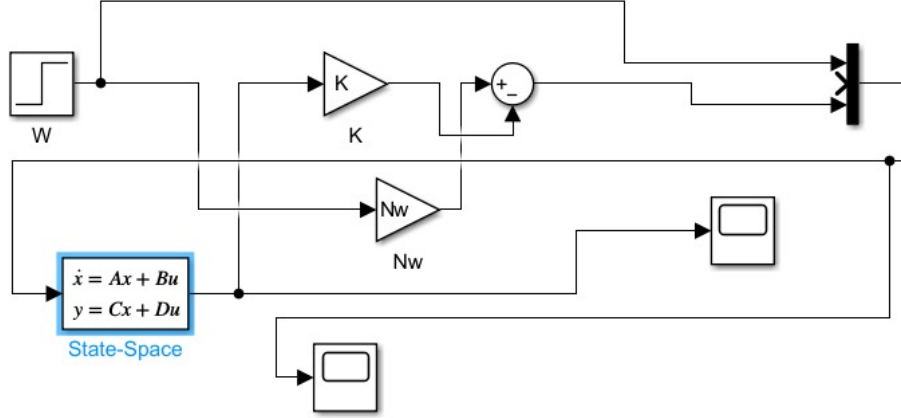


Figure 8: Simulink block diagram

11 Passive suspension with real road generator

11.1 Road input definition for suspension simulation

In a quarter-car (or half/full-car) suspension model, the road acts as a base displacement at the tire-road contact point. We denote this road input by $W(t)$ in meters. Real road geometry is typically described in the spatial domain as a vertical displacement profile $z(x)$ (meters) as a function of longitudinal distance x (meters). Assuming the vehicle travels at a (locally) constant forward speed v (m/s), distance and time are related by

$$x = vt, \quad (16)$$

therefore the time-domain road input applied to the suspension becomes

$$W(t) = z(vt). \quad (17)$$

Hence, the problem reduces to generating a realistic random spatial road profile $z(x)$ and mapping it into time using (17).

11.2 ISO 8608 road roughness model in the spatial frequency domain

ISO 8608 provides a method to classify road profiles according to a specific fitting process. Starting from the equation provided by ISO 8608 standard for the calculation of fitted power spectral density (PSD) of profile elevations, it is possible to generate random road profiles having different geometric characteristics, thus belonging to the desired road profile class. ISO 8608 characterizes road roughness statistically through the spatial power spectral density (PSD) of the displacement profile, denoted $G_d(n)$, as a function of

spatial frequency n (cycles/m). For a stationary, zero-mean profile, the variance of road height within a spatial-frequency band $[n_{\min}, n_{\max}]$ is

$$\sigma_z^2 = \int_{n_{\min}}^{n_{\max}} G_d(n) dn. \quad (18)$$

A widely used ISO 8608 representation is a power-law PSD of the form

$$G_d(n) = G_d(n_0) \left(\frac{n}{n_0} \right)^{-w}, \quad (19)$$

where $n_0 = 0.1$ cycles/m is the reference spatial frequency, $G_d(n_0)$ is the roughness level (used for ISO road class bands), and w is the waviness exponent (often taken near 2 for artificial profile synthesis).

11.3 Spectral synthesis of the spatial road profile

To generate a random profile consistent with the target PSD in (19), a spectral (harmonic) representation is used:

$$z(x) = \sum_{i=1}^N A_i \cos(2\pi n_i x + \phi_i), \quad (20)$$

where:

- n_i are discrete spatial frequencies spanning $[n_{\min}, n_{\max}]$ with spacing Δn ,
- $\phi_i \sim \mathcal{U}[0, 2\pi]$ are independent random phases, and
- A_i are amplitudes chosen to match the prescribed PSD.

11.3.1 Derivation of the amplitude formula

The phases are selected as independent and uniformly distributed so that the process is (approximately) stationary and zero-mean:

$$\mathbb{E}[z(x)] = 0. \quad (21)$$

The mean-square value is

$$\mathbb{E}[z^2(x)] = \mathbb{E} \left[\left(\sum_{i=1}^N A_i \cos(2\pi n_i x + \phi_i) \right)^2 \right]. \quad (22)$$

Expanding the square yields self-terms and cross-terms. Due to independence and uniformity of the phases, cross-terms average to (approximately) zero, while each self-term satisfies

$$\mathbb{E}[\cos^2(2\pi n_i x + \phi_i)] = \frac{1}{2}. \quad (23)$$

Thus,

$$\mathbb{E}[z^2(x)] \approx \sum_{i=1}^N \frac{A_i^2}{2}. \quad (24)$$

On the other hand, the target PSD implies (discretizing (18)):

$$\sigma_z^2 \approx \sum_{i=1}^N G_d(n_i) \Delta n. \quad (25)$$

Matching (24) and (25) term-by-term gives

$$\frac{A_i^2}{2} = G_d(n_i) \Delta n \quad \Rightarrow \quad A_i = \sqrt{2 G_d(n_i) \Delta n}. \quad (26)$$

Equation (26) is the key result used in the implementation: it guarantees that, in expectation, the synthesized profile has the desired PSD over the selected band.

11.4 Conversion to the time-domain road input $W(t)$

Once $z(x)$ is synthesized, the time-domain road input for simulation is obtained from (17). For spatial sampling step Δx , the corresponding time step is

$$\Delta t = \frac{\Delta x}{v}. \quad (27)$$

A useful interpretation is that spatial frequency n maps to temporal frequency f (Hz) according to

$$f = vn, \quad (28)$$

which means that increasing vehicle speed shifts the same spatial roughness content into higher excitation frequencies, consistent with real vehicle operation.

11.5 Parameter selection and realism justification

The adopted frequency band $[n_{\min}, n_{\max}]$ determines the wavelengths included in the road profile:

$$\lambda_{\max} = \frac{1}{n_{\min}}, \quad \lambda_{\min} = \frac{1}{n_{\max}}. \quad (29)$$

For example, choosing $n_{\min} = 0.01$ cycles/m and $n_{\max} = 10$ cycles/m corresponds to wavelengths from 100 m down to 0.1 m, covering long undulations as well as short roughness components relevant to ride and wheel-hop excitation.

Expected RMS roughness (sanity check). Using (19) in (18), for $w \neq 1$:

$$\sigma_z^2 = \int_{n_{\min}}^{n_{\max}} G_d(n) dn = G_d(n_0) n_0^w \frac{n_{\max}^{1-w} - n_{\min}^{1-w}}{1-w}. \quad (30)$$

For the common case $w = 2$:

$$\sigma_z^2 = G_d(n_0) n_0^2 \left(\frac{1}{n_{\min}} - \frac{1}{n_{\max}} \right). \quad (31)$$

This provides a quick check that the generated roughness magnitude is physically plausible (typically millimeter-level RMS for smoother roads and larger for rougher classes), and that the simulation is not being driven by unrealistically large displacements.

Why this method is realistic enough for active-suspension evaluation. The ISO-PSD synthesis approach is suitable for controller and ride-dynamics evaluation because:

- **Standard-based statistics:** the generated profile matches ISO 8608 second-order statistics (PSD level and slope) over a defined frequency band.
- **Broadband excitation:** it excites both low-frequency body modes and higher-frequency wheel-hop dynamics, which is essential for evaluating active suspension performance beyond simple step/bump tests.
- **Correct speed dependence:** the mapping $W(t) = z(vt)$ and $f = vn$ reproduces the physical effect of speed on excitation frequency content.

11.6 MATLAB Implementation and Simulation Results

The obtained results demonstrate that the designed control system performs effectively. The controller successfully regulates the vertical displacement y_1 around zero, which significantly enhances ride comfort and vehicle stability.

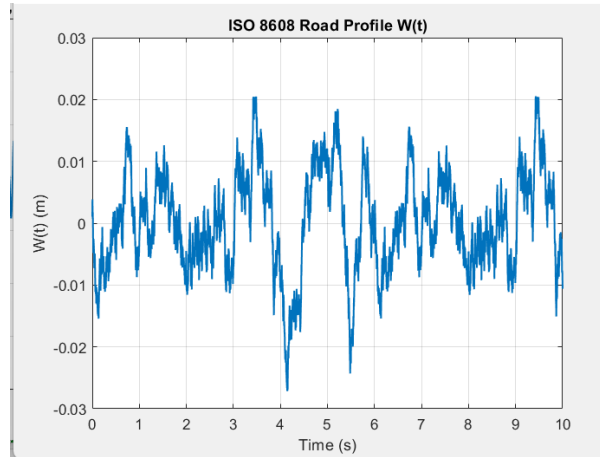


Figure 9: Road profile

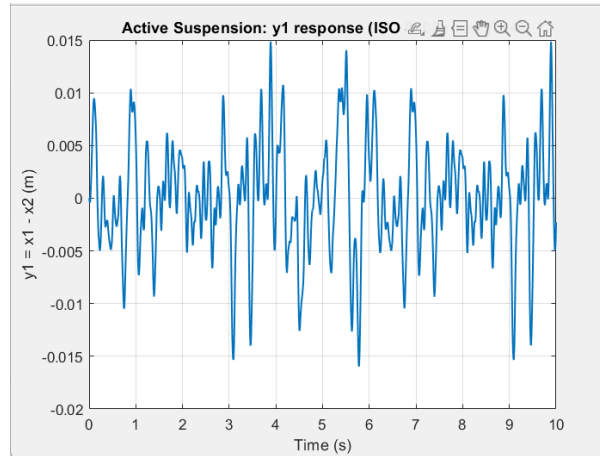


Figure 10: Active suspension: y_1 response under ISO 8608 road profile using RK4 integration

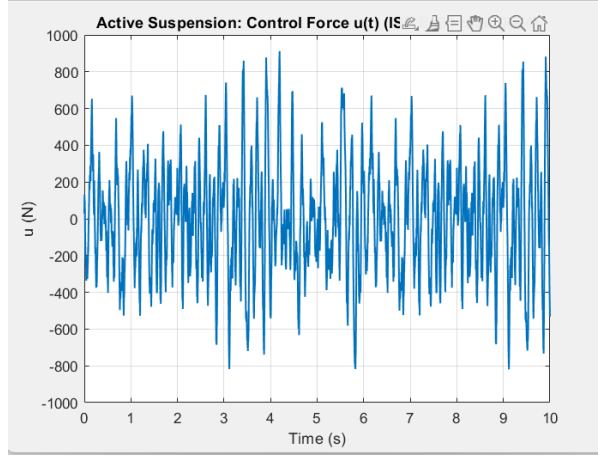


Figure 11: Active suspension: control force $u(t)$ under ISO 8608 road profile

11.7 MATLAB Implementation with Ramp and Bump Road Inputs

The road disturbance input $w(t)$ is modeled as a saturating ramp followed by a half-sine bump:

$$w(t) = W_{\text{final}} r(t) + A_{\text{bump}} b(t) \quad (32)$$

$$r(t) = \begin{cases} 0, & t < t_{r0} \\ \frac{t - t_{r0}}{T_{\text{ramp}}}, & t_{r0} \leq t \leq t_{r0} + T_{\text{ramp}}, \quad T_{\text{ramp}} > 0 \\ 1, & t > t_{r0} + T_{\text{ramp}}, \quad T_{\text{ramp}} > 0 \\ \mathbb{K}(t \geq t_{r0}), & T_{\text{ramp}} \leq 0 \end{cases} \quad (33)$$

$$b(t) = \begin{cases} \sin\left(\frac{\pi(t - t_{b0})}{T_{\text{bump}}}\right), & t_{b0} \leq t \leq t_{b0} + T_{\text{bump}} \\ 0, & \text{otherwise} \end{cases} \quad (34)$$

This combined road profile represents a gradual elevation change followed by a localized bump disturbance.

The following figures illustrate the system response when subjected to ramp and bump road disturbances.

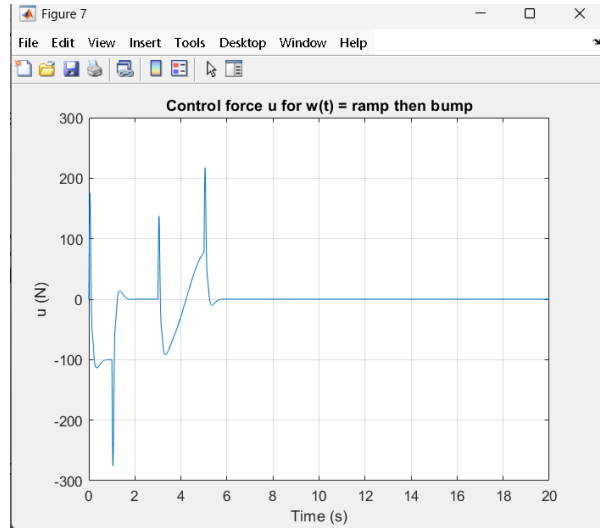


Figure 12: Control force $u(t)$ response

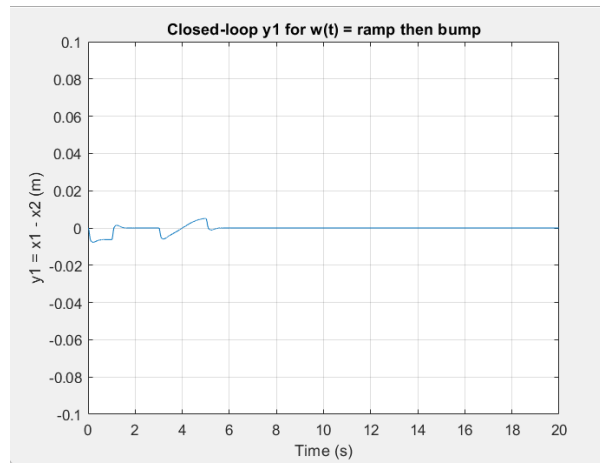


Figure 13: y_1 response

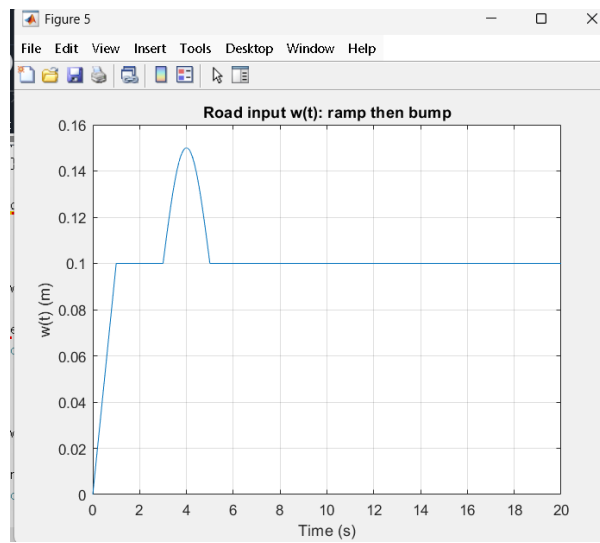


Figure 14: System vertical displacement response under combined ramp and bump disturbances

12 LQR State-Feedback Design

12.1 Objective and requirements (Project)

The active suspension problem in this project is posed as a disturbance-rejection control task: the road input $w(t)$ (terrain disturbance) excites the quarter-car dynamics, and the actuator force $u(t)$ is used to reduce undesirable vibration and suspension motion. The project statement defines the primary objective as monitoring and controlling the suspension deflection ($x_1 - x_2$) while rejecting road disturbances and maintaining stability under bumps/holes in the road. In Parts (8)–(10), the controller must satisfy the transient requirement for a 0.1 m step road disturbance and must be verified in Simulink by plotting the time responses of the output y_1 and the control signal u . In Task (10) specifically, Parts (8) and (9) are repeated using an LQR design and the selection of the weighting matrices must be justified [7].

12.2 LQR formulation and control law

In Part (8) we previously designed a full-state feedback controller (e.g., via pole placement) to meet the disturbance transient requirements. In Task (10), we instead use an LQR (Linear Quadratic Regulator), which computes an *optimal* state feedback gain by minimizing a quadratic performance index while balancing performance vs. control effort.

A convenient (and commonly used) LQR formulation begins by defining a quadratic cost over time:

$$J = \int_0^\infty (\mathbf{y}^\top \mathbf{Q}_y \mathbf{y} + \mathbf{u}^\top \mathbf{R}_y \mathbf{u}) dt, \quad (35)$$

where $\mathbf{Q}_y \succeq 0$ penalizes performance outputs and $\mathbf{R}_y \succ 0$ penalizes actuator effort. This can be transformed into the standard LQR form in terms of states:

$$J = \int_0^\infty (\mathbf{x}^\top \mathbf{Q} \mathbf{x} + \mathbf{u}^\top \mathbf{R} \mathbf{u}) dt, \quad \mathbf{Q} = \mathbf{C}^\top \mathbf{Q}_y \mathbf{C}, \quad \mathbf{R} = \mathbf{R}_y + \mathbf{D}^\top \mathbf{Q}_y \mathbf{D}, \quad (36)$$

(with $\mathbf{D} = 0$ here because y_1 has no direct feedthrough from u). The optimal control law is:

$$u(t) = -\mathbf{K}\mathbf{x}(t), \quad (37)$$

where $\mathbf{K}$ is computed from the solution $\mathbf{P}$ of the continuous-time algebraic Riccati equation (ARE):

$$\mathbf{K} = \mathbf{R}^{-1} \mathbf{B}_u^\top \mathbf{P}, \quad (38)$$

$$\mathbf{A}^\top \mathbf{P} + \mathbf{P} \mathbf{A} - \mathbf{P} \mathbf{B}_u \mathbf{R}^{-1} \mathbf{B}_u^\top \mathbf{P} + \mathbf{Q} = \mathbf{0}. \quad (39)$$

This procedure is standard in optimal control design and is implemented in MATLAB with `lqr()` (or equivalently by solving the ARE and forming $\mathbf{K}$) [8].

12.3 How the LQR weights Q and R were selected (and tuned)

Task (10) explicitly requires an explanation of how the weighting matrices are selected. The key idea is that Q and R encode the design priorities:

- Larger Q (relative to R) $\Rightarrow$ more aggressive regulation (smaller vibration/deflection), but typically larger control force peaks $u(t)$.

- Larger R (relative to Q) $\Rightarrow$ smaller actuator effort, but slower disturbance rejection.

A systematic and physically interpretable approach is to weight the *actual performance variable* required by the project, namely $y_1 = x_1 - x_3$. Because $y_1 = \mathbf{C}\mathbf{x}$, a scalar penalty q_y on y_1^2 yields:

$$\mathbf{Q} = \mathbf{C}^\top q_y \mathbf{C} = q_y \begin{bmatrix} 1 & 0 & -1 & 0 \end{bmatrix}^\top \begin{bmatrix} 1 & 0 & -1 & 0 \end{bmatrix}. \quad (40)$$

This directly penalizes suspension travel (deflection), which is the output that must be plotted and discussed in the Task (10) deliverables [7].

Initial scaling from design specs. The disturbance test specified in the project is a 0.1 m road step. A natural first choice is to set the penalty such that y_1 deviations of order 0.1 m are considered “large”. One practical guideline is to set $q_y \propto 1/(y_{1,\max})^2$ (so exceeding the allowable deflection is heavily penalized), and set R based on an acceptable control-force magnitude:

$$q_y \sim \frac{1}{(0.1)^2}, \quad R \sim \frac{1}{u_{\max}^2}. \quad (41)$$

After this initial scaling, the weights must be fine-tuned until the time-domain requirements are met. This is consistent with the standard LQR tuning workflow: choose Q and R from specifications, then fine-tune based on simulation results [8].

Final tuned weights used in our simulation. Starting from the above principle and tuning to satisfy the 0.1 m step requirement in Part (8), we selected:

$$q_y = 3.0 \times 10^4, \quad R = 1.0 \times 10^{-6}, \quad (42)$$

which produced an LQR gain (for our parameter set):

$$\mathbf{K}_{\text{LQR}} = \begin{bmatrix} 1.579 \times 10^5 & 6.057 \times 10^3 & -1.024 \times 10^5 & -1.359 \times 10^3 \end{bmatrix}. \quad (43)$$

(If the parameter set differs, the same tuning method applies; the final numerical gain will change.)

12.4 Road/Terrain Disturbance Modeling as a General Input $W(t)$

12.4.1 What W means physically: road profile as height vs. distance

A road surface is most naturally described as a *spatial* height profile $z(x)$ [m] measured along the road distance x [m]. The quarter-car model uses the road displacement at the tire contact patch as the disturbance input, so the disturbance is

$$W(t) \triangleq z(x(t)), \quad (44)$$

where $x(t)$ is the longitudinal position of the vehicle. For constant forward speed v ,

$$x(t) = vt \quad \Rightarrow \quad W(t) = z(vt). \quad (45)$$

Therefore, the **same road** $z(x)$ produces **different time signals** $W(t)$ at different speeds v .

Discrete implementation of a measured profile. If $z(x)$ is measured on a spatial grid $\{x_i, z_i\}$, then in discrete-time simulation ($t_k = k\Delta t$),

$$W[k] = z(vk\Delta t) \approx \text{interp}(\{x_i, z_i\}, vk\Delta t), \quad (46)$$

using interpolation (linear/spline). This step is essential: without (45)–(46), a “terrain” profile defined in space cannot be injected correctly into a time-domain simulator.

12.4.2 How $W(t)$ enters the suspension dynamics

With tire damping neglected ($B_2 \approx 0$), the quarter-car state model is

$$\dot{x}(t) = Ax(t) + B_u u(t) + B_w W(t), \quad y_1(t) = Cx(t), \quad C = \begin{bmatrix} 1 & 0 & -1 & 0 \end{bmatrix}. \quad (47)$$

Thus, road excitation is an *exogenous input channel*. For any admissible $W(t)$, the solution is

$$x(t) = e^{At}x(0) + \int_0^t e^{A(t-\tau)} (B_u u(\tau) + B_w W(\tau)) d\tau, \quad (48)$$

which generally requires numerical evaluation for arbitrary $W(\cdot)$.

If tire damping is included ($B_2 \neq 0$). The wheel equation contains $-B_2(\dot{x}_2 - \dot{W})$, so the disturbance effectively enters through $(W, \dot{W})$. In that case, either:

- provide $\dot{W}(t)$ explicitly, or
- ensure $W(t)$ is differentiable (e.g., generated by smooth functions/filters) and compute $\dot{W}$ numerically.

12.4.3 Numerical simulation used in this project: ZOH discretization

We simulated in discrete time with sample time Δt and a zero-order hold (ZOH) assumption on the inputs over each interval $[k\Delta t, (k+1)\Delta t)$:

$$u(t) \approx u[k], \quad W(t) \approx W[k], \quad t \in [k\Delta t, (k+1)\Delta t). \quad (49)$$

Under ZOH, the exact discrete-time update is

$$x[k+1] = A_d x[k] + B_{u,d} u[k] + B_{w,d} W[k], \quad (50)$$

where

$$A_d = e^{A\Delta t}, \quad B_{(\cdot),d} = \int_0^{\Delta t} e^{A\tau} d\tau B_{(\cdot)}. \quad (51)$$

This is the mathematical reason we can inject arbitrary terrain sequences $W[k]$: once (50) is defined, the simulator simply steps forward using the supplied samples $W[k]$.

12.4.4 Deterministic terrain components (waves and bumps)

We model terrain as a superposition of components:

$$W(t) = W_{\text{wave}}(t) + W_{\text{bump}}(t) + W_{\text{rough}}(t), \quad (52)$$

which is valid because the plant is linear in inputs.

(1) Wavy road (low-frequency undulation).

$$W_{\text{wave}}(t) = A_w \sin(2\pi f_w t) \alpha(t), \quad \alpha(t) = \min\left(\frac{t}{T_f}, 1\right). \quad (53)$$

The envelope $\alpha(t)$ ensures a smooth start (no artificial discontinuity at $t = 0$).

(2) Localized bump/pothole events. A smooth bump can be modeled in *space* as a Gaussian-shaped elevation:

$$z_{\text{bump}}(x) = A_b \exp\left(-\frac{(x - x_b)^2}{2\sigma_x^2}\right), \quad (54)$$

which becomes a time signal under constant speed $x = vt$:

$$W_{\text{bump}}(t) = A_b \exp\left(-\frac{(vt - x_b)^2}{2\sigma_x^2}\right) = A_b \exp\left(-\frac{(t - t_b)^2}{2\sigma_t^2}\right), \quad (55)$$

where $t_b = x_b/v$ and $\sigma_t = \sigma_x/v$. Multiple bumps are modeled by summation:

$$W_{\text{bump}}(t) = \sum_{i=1}^{N_b} A_i \exp\left(-\frac{(t - t_i)^2}{2\sigma_i^2}\right). \quad (56)$$

This explicitly shows how “terrain features in space” map into a time-domain input for simulation.

12.4.5 Stochastic roughness component (math-correct noise shaping)

Using raw white noise directly as displacement is not physically meaningful unless it is band-limited/shaped. A standard approach is to generate *colored* roughness by filtering white noise.

Continuous-time shaping filter model. Let $\eta(t)$ be unit white noise. Define a stable shaping filter

$$\dot{x}_r(t) = A_r x_r(t) + B_r \eta(t), \quad W_{\text{rough}}(t) = C_r x_r(t), \quad (57)$$

so that W_{rough} is band-limited. A simple first-order low-pass choice is:

$$H(s) = \frac{\omega_c}{s + \omega_c}, \quad (58)$$

which corresponds to the state equation

$$\dot{W}_{\text{rough}}(t) = -\omega_c W_{\text{rough}}(t) + \omega_c \sigma_n \eta(t). \quad (59)$$

Discrete-time implementation (exact recursion). Discretizing (59) with sample time Δt gives an AR(1) recursion:

$$W_{\text{rough}}[k] = a W_{\text{rough}}[k-1] + b \xi[k], \quad (60)$$

where $\xi[k] \sim \mathcal{N}(0, 1)$, and

$$a = e^{-\omega_c \Delta t}, \quad b = \sigma_n \sqrt{1 - a^2}. \quad (61)$$

The choice (61) ensures W_{rough} has steady-state standard deviation approximately σ_n while remaining band-limited by ω_c and the sampling Δt . This also prevents aliasing artifacts when simulating rough roads.

12.4.6 Final discrete terrain sequence used by the plant

Combining (52) and sampling at $t_k = k\Delta t$ yields

$$W[k] = W_{\text{wave}}(k\Delta t) + W_{\text{bump}}(k\Delta t) + W_{\text{rough}}[k], \quad (62)$$

which is then injected into the discrete-time plant update (50). Therefore, the ability to use “arbitrary terrain” is not merely changing a block input; it is the combination of:

1. defining terrain physically in space $z(x)$ and converting it to time $W(t) = z(vt)$,
2. generating deterministic and stochastic components with well-defined bandwidth,
3. sampling to obtain $W[k]$ consistent with the discrete simulation step,
4. propagating the dynamics numerically via (50).

12.5 Simulink implementation (Task 9 repeated with LQR)

To verify the design requirements (Task 9) using an LQR controller (), the Simulink model was organized into four main parts:

1. **Road Generator:** produces $w(t)$ either as a 0.1 m step or as the terrain profile in (62).
2. **Plant (Quarter-car State-Space):** implements (1)–(40) via a state-space block using $\mathbf{A}$, $\mathbf{B}_u$, and $\mathbf{B}_w$.
3. **LQR Controller:** computes $u(t) = -\mathbf{K}_{\text{LQR}}\mathbf{x}(t)$ (Gain block).
4. **Outputs/Logging:** computes $y_1 = x_1 - x_3$ and logs both $y_1(t)$ and $u(t)$ for plotting.

A fixed-step simulation was used to ensure repeatability and to match how a digital controller would run in practice.

Following is the Simulink block diagram:

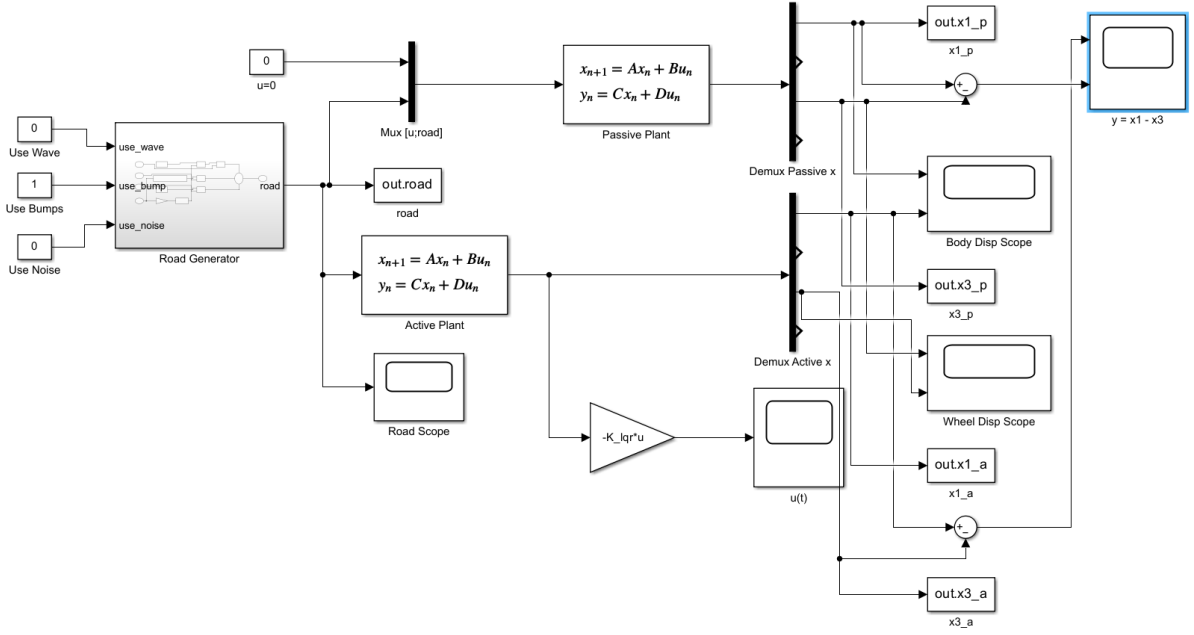


Figure 15: Simulink Block Diagram (task 10)

With the road generator:

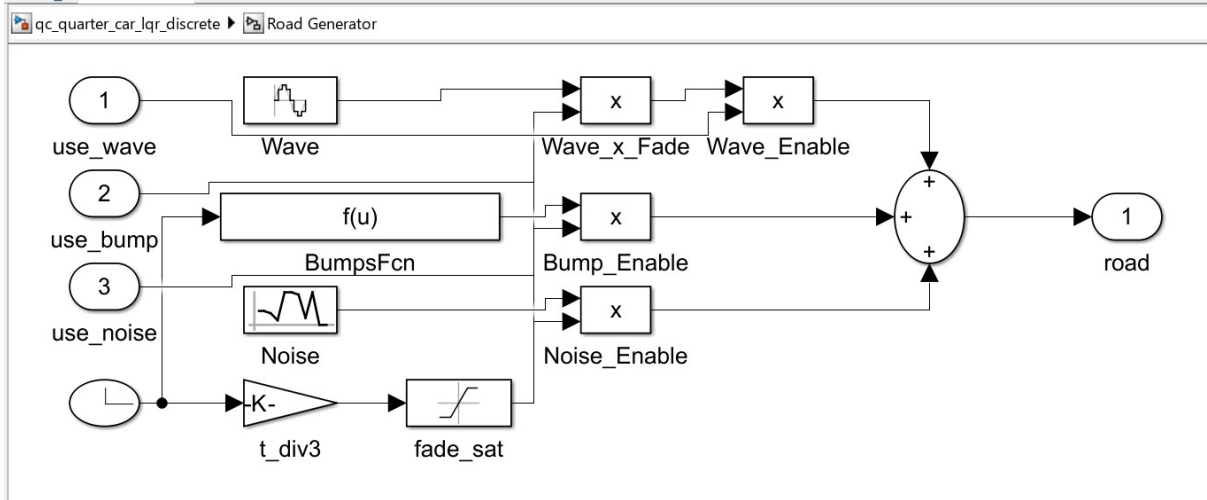


Figure 16: Road Generator Block Diagram Within Simulink Implementation

This diagram is very nice, since we can control what kind of terrain we want by switching between the 1s and 0s in the far-left blocks.

12.6 Results and discussion

Step disturbance verification ($w = 0.1$ m). Figures 22–27 show the required Task (10) plots: $y_1(t)$ and $u(t)$ for a 0.1 m road step. With the tuned weights in (42), the closed-loop response:

- kept the body motion within the ± 0.1 m requirement,
- reduced the suspension deflection peak to within ± 0.1 m,

- and eliminated oscillations well within the 5 s requirement.

The control signal $u(t)$ exhibits a large initial corrective force (to counteract the road step), then decays quickly as the system reaches a smooth steady condition. This behavior is expected: LQR explicitly balances vibration reduction against actuator effort through the Q/R trade-off [8].

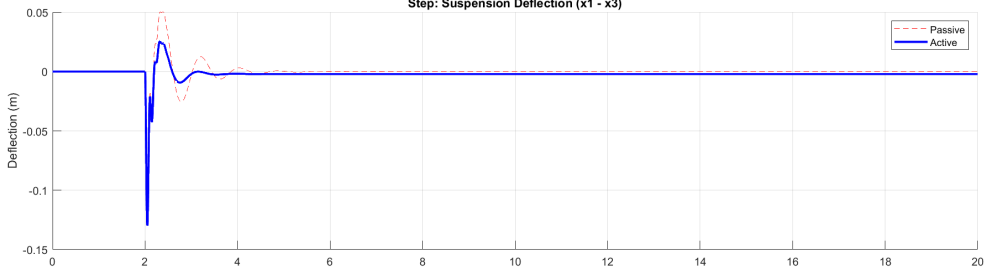


Figure 17: : LQR response of $y_1 = x_1 - x_3$ to a 0.1 m step road disturbance. (MATLAB)

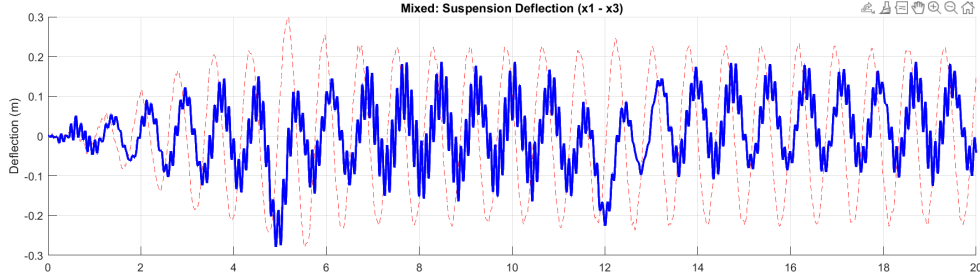


Figure 18: : LQR response of $y_1 = x_1 - x_3$ to a mixed (resonance wave + gravel + bumps) road disturbance. The dashed passive line makes it hard to smoother. (MATLAB)

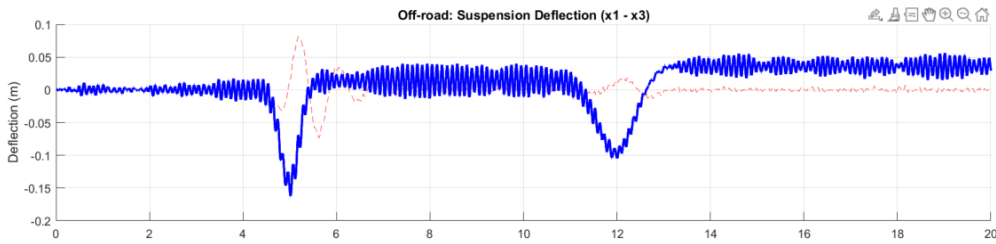


Figure 19: LQR response $y_1 = x_1 - x_3$ for offroad terrain (gravel + bumps)

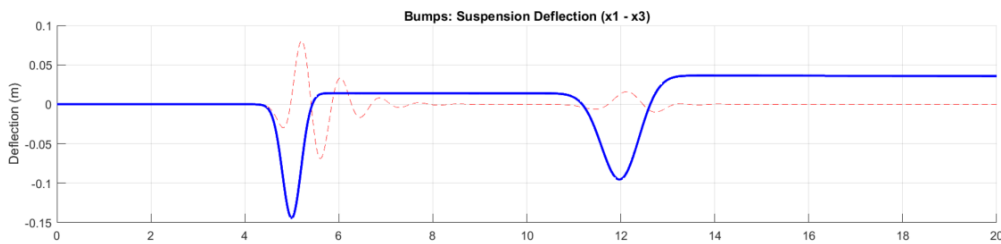


Figure 20: LQR suspension deflection response for bumpy road (harsh bump → soft bump)

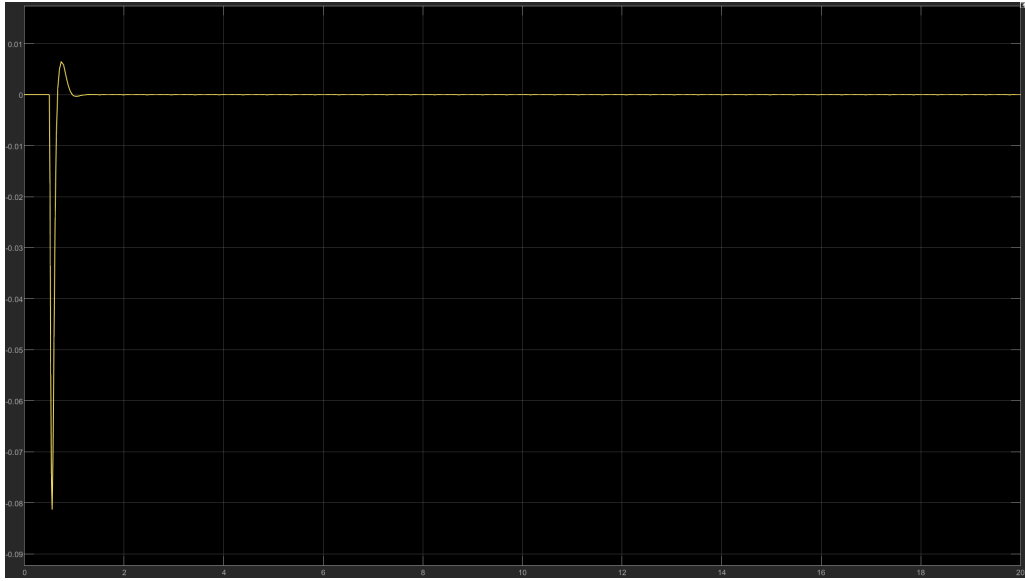


Figure 21: : LQR response of $y_1 = x_1 - x_3$ to a 0.1 m step road disturbance. (Simulink)

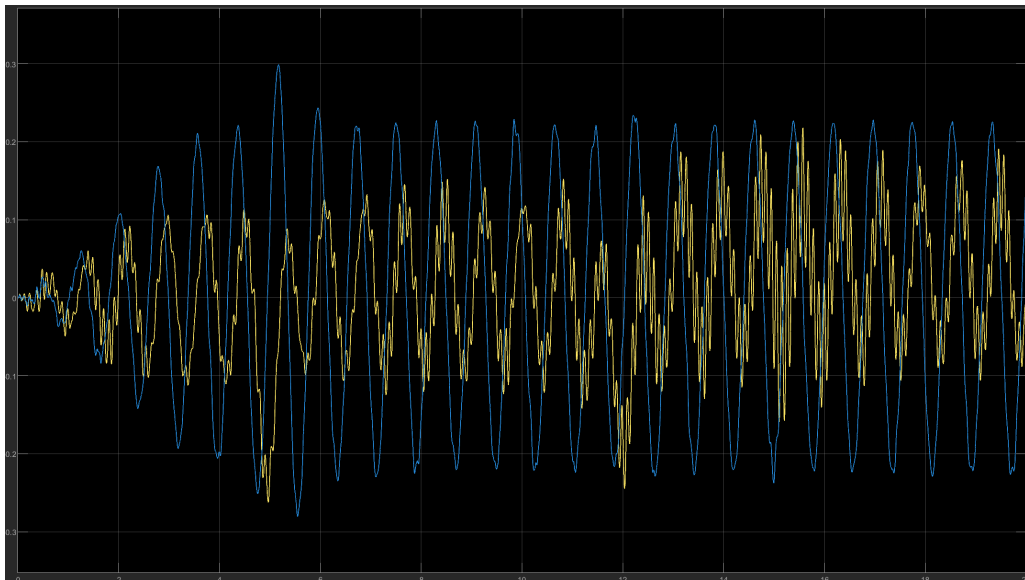


Figure 22: : LQR response of $y_1 = x_1 - x_3$ to a mixed (resonance wave + gravel + bumps) road disturbance. (Simulink)

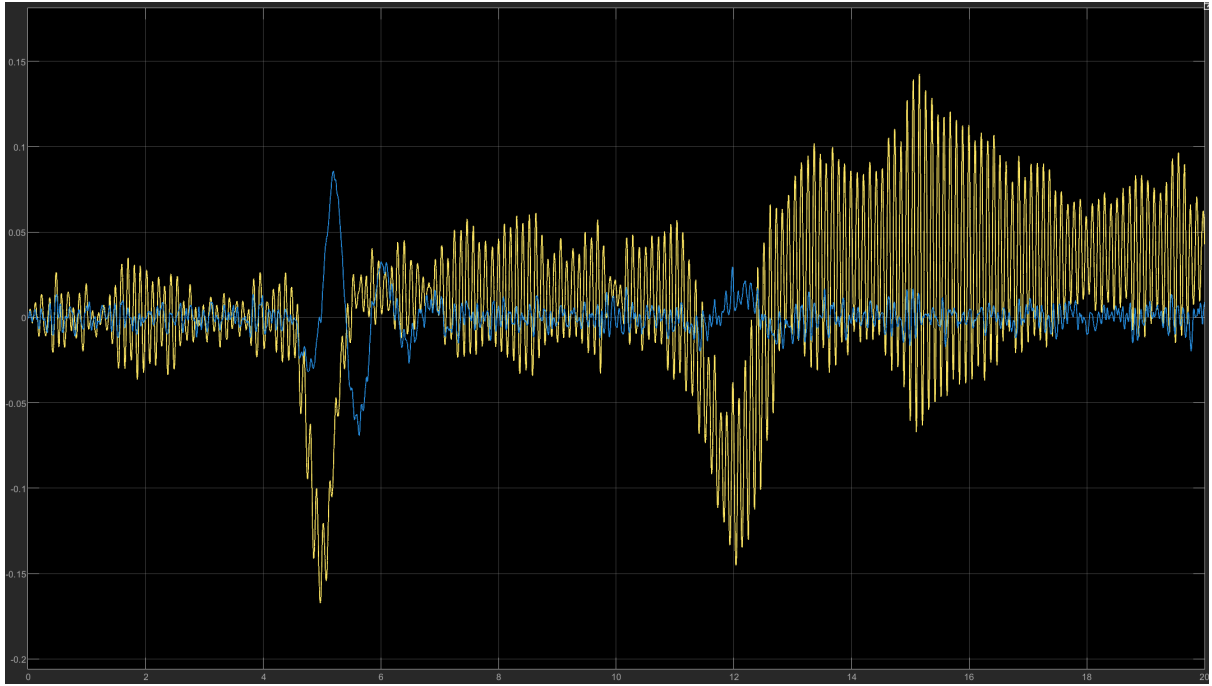


Figure 23: LQR response of $y_1 = x_1 - x_3$ to an offroad (gravel + bumps) road disturbance. (Simulink)

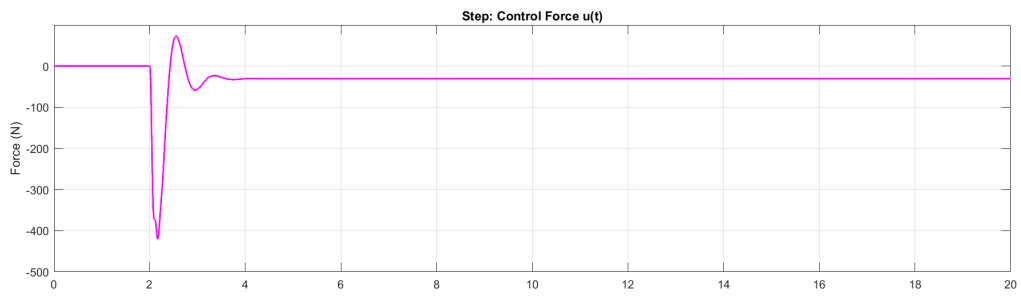


Figure 24: : LQR control force $u(t)$ for the same 0.1 m step disturbance. (Unit, MATLAB)

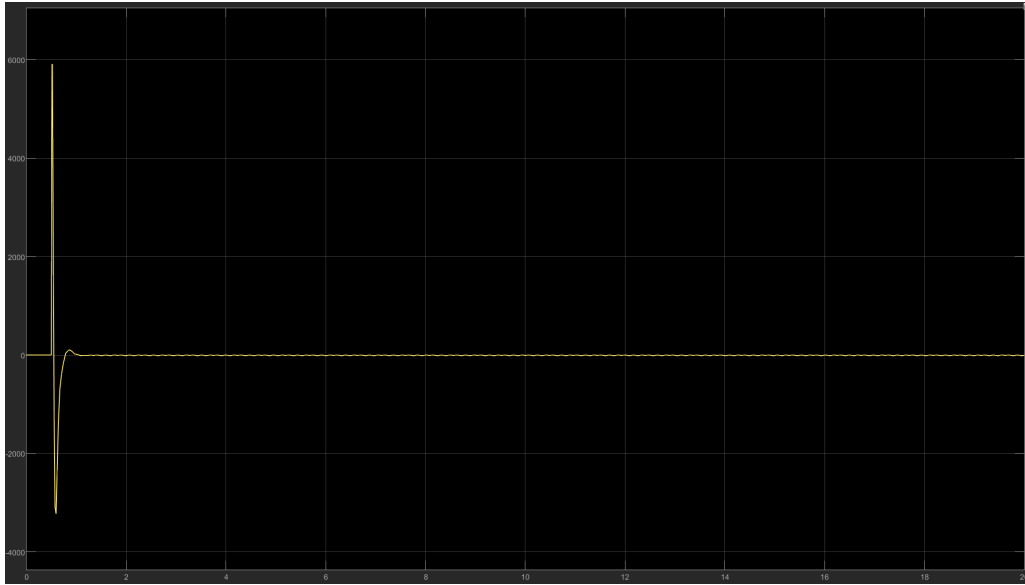


Figure 25: LQR control force $u(t)$ for the same 0.1 m step disturbance.(Unit, Simulink)
(It's zoomed in hence why the simulink one looks thinner than MATLAB.)

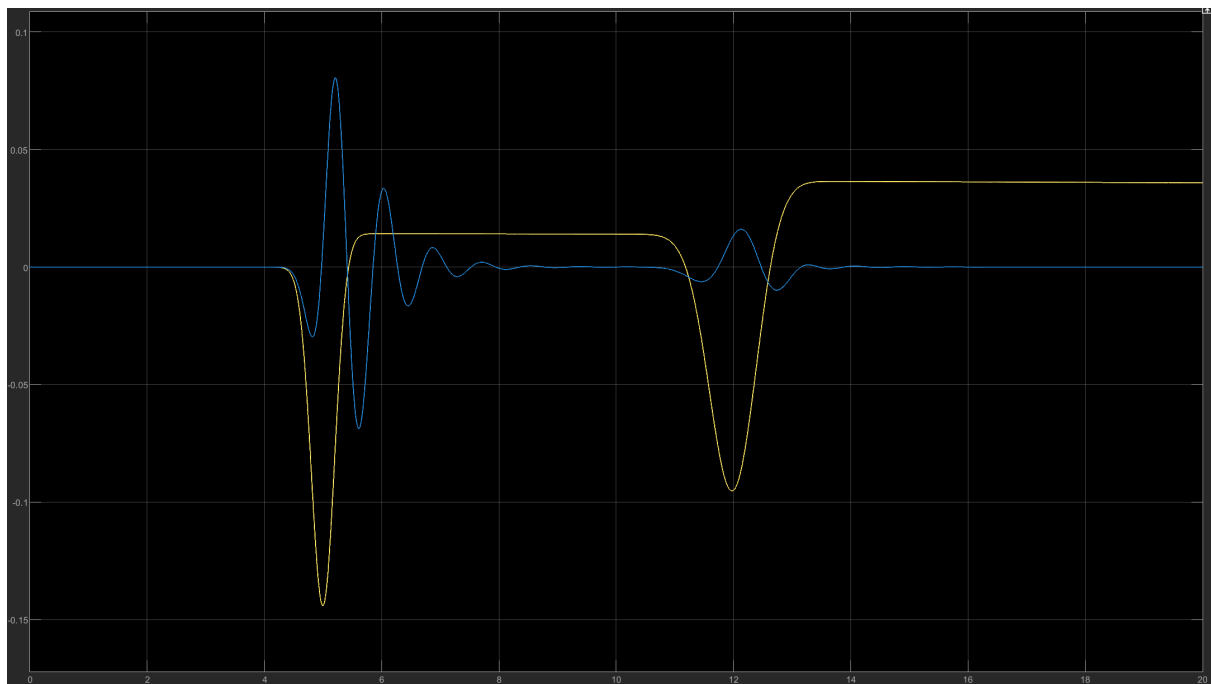


Figure 26: LQR suspension deflection response for bumpy road(harsh bump $\rightarrow$ soft bump)
(Simulink)

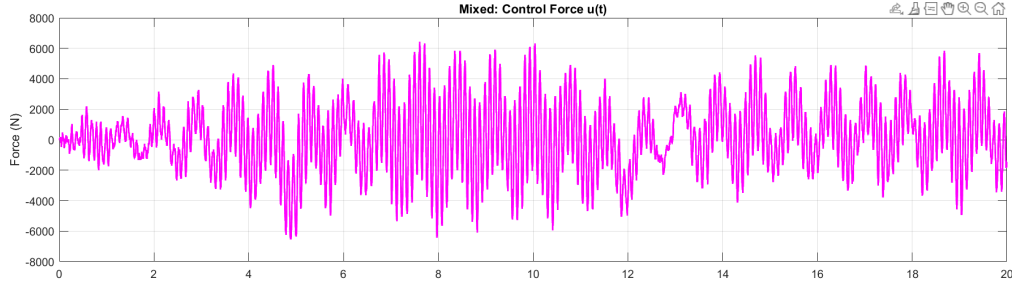


Figure 27: LQR control force $u(t)$ for mixed terrain disturbance.

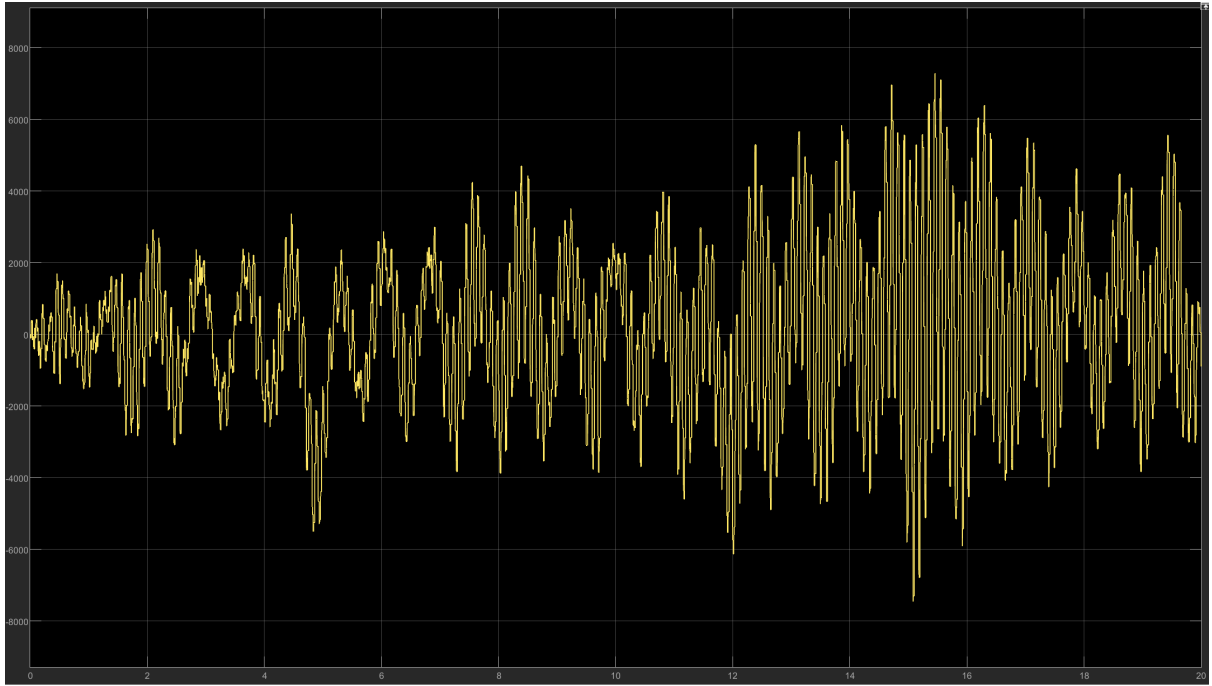


Figure 28: LQR control force $u(t)$ for mixed terrain disturbance.(Simulink) (Bump & noise locations are random so it won't be the exact same as MATLAB, although we can see the same pattern/trend)

Interpretation of the LQR outcome. Compared to a pole-placement design, LQR provides a systematic method to compute $\mathbf{K}$ by minimizing a well-defined cost function rather than manually selecting closed-loop poles. The main practical advantage is that the tuning knobs have direct meaning: penalizing y_1 more (increasing q_y) pushes the controller to reduce suspension travel more aggressively, while increasing R reduces the peak force demand but typically slows the transient response [8].

References

- [1] R. Rajamani, *Vehicle Dynamics and Control*, 2nd ed. Springer, 2012.
- [2] D. Hrovat, "Survey of advanced suspension developments and related optimal control applications," *Automatica*, vol. 33, no. 10, pp. 1781–1817, 1997.
- [3] MathWorks, *Control System Toolbox Documentation*, accessed 2025.

- [4] MathWorks, *Simulink Documentation*, accessed 2025.
- [5] University of Michigan, “Quarter-car Suspension System Modeling,” *Control Tutorials for MATLAB and Simulink (CTMS)*, 2025. Available: <https://ctms.engin.umich.edu/CTMS/index.php?example=Suspension§ion=SystemModeling>
- [6] MathWorks, “Linear Quadratic Regulator (LQR),” *MathWorks Control System Toolbox Documentation*, 2025. Available: https://www.mathworks.com/help/control/ref/lqi.lqr.html?utm_source=chatgpt.com
- [7] SPC302 Course, *Project Fall 2025: An Introduction to Linear Quadratic Regulators(LQR)*, (Fall 2025), 2025.
- [8] Y. Lu, C. Sun, and A. Khajepour, *Active and Semi-active Suspension Systems: Modeling, Control, and Applications*, Springer Nature Switzerland, 2025. DOI: 10.1007/978-3-031-73924-8.

A Appendix A: MATLAB Scripts

A.1 MainSimulation.m

Listing 1: MainSimulation.m

```

1 %% ACTIVE SUSPENSION      ACTIVE PART ONLY
2
3 clear; close all; clc
4
5 %% =====
6 % 0) Parameters
7 %% =====
8 m1 = 290;          % kg (sprung mass)
9 m2 = 59;          % kg (unsprung mass)
10 k1 = 16000;       % N/m (suspension spring)
11 k2 = 190000;      % N/m (tire stiffness)
12 b1 = 1000;        % N*s/m (suspension damper)
13 b2 = 0;           % N*s/m (tire damper)  <-- keep 0 so road input
    is W only (no Wdot)
14
15 W_step = 0.1;     % [m] step road disturbance
16
17 if b2 ~= 0
18     warning('b2 is not zero. Your Bw below assumes road input is
19         W only. Set b2=0 or re-derive state-space with Wdot.');
```

end

```

20
21 %% =====
22 % 1) State-space model (x = [x1; x1dot; x2; x2dot])
23 %% =====
24 A = [ 0          1          0          0;
25       -k1/m1     -b1/m1      k1/m1      b1/m1;
26       0          0          0          1;
27       k1/m2      b1/m2     -(k1+k2)/m2  -(b1+b2)/m2 ];
28
29 Bu = [0; 1/m1; 0; -1/m2];    % control force u
30 Bw = [0; 0; 0; k2/m2];      % road displacement W(t) (valid
    cleanly when b2=0)
31
32 C = [1 0 -1 0];             % y1 = x1 - x2 (suspension deflection
    )
33 Du = 0;
34
35 n = size(A,1);
36
37 %% =====
38 % 2) Controllability check (needed for place)
39 %% =====
40 Co = ctrb(A, Bu);
41 rCo = rank(Co);
42
```

```

43 fprintf('\n=== Controllability ===\n');
44 fprintf('rank(ctrb(A,Bu)) = %d, n = %d\n', rCo, n);
45
46 if rCo < n
47     error("System NOT controllable w.r.t. u: rank=%d < %d. Can't
48         use place().", rCo, n);
49 end
50 %% =====
51 % 3) State feedback design (u = -Kx + Nw*W)
52 %% =====
53
54 p_des = [-7.91+5.90j, -7.91-5.90j, -20.19+45.53j, -20.19-45.53j];
55
56 K = place(A, Bu, p_des);
57
58 Acl = A - Bu*K;
59
60 % Disturbance feedforward to make y_ss 0 for constant W (if
61     possible)
62 den = C*(Acl\Bu);
63 if abs(den) < 1e-12
64     warning('C*(Acl\Bu) is ~0 -> setting Nw=0 (no road
65         feedforward).');
66     Nw = 0;
67 else
68     Nw = -(C*(Acl\Bw)) / den; % scalar
69 end
70
71 Bcl = Bw + Bu*Nw;
72
73 fprintf('\n=== Controller ===\n');
74 fprintf('K = [%.6f %.6f %.6f %.6f]\n', K);
75 fprintf('Nw = %.6f\n', Nw);
76
77 %% =====
78 % 4) STEP road simulation (lsim)
79 %% =====
80 % Closed-loop from W -> outputs [y1; u]
81 sys_cl = ss(Acl, Bcl, [C; -K], [0; Nw]);
82
83 t_step = 0:0.001:10;
84 w_vec = W_step*ones(size(t_step));
85
86 yout = lsim(sys_cl, w_vec, t_step);
87 y1_step = yout(:,1);
88 u_step = yout(:,2);
89
90 peakY = max(abs(y1_step));
91 TsY = settling_time_to_zero(t_step, y1_step, 0.02);

```



```

91 fprintf('\n=== STEP road results (W = 0.1 m) ===\n');
92 fprintf('Peak |y1| = %.4f m (require <= 0.1 m)\n', peakY);
93 fprintf('SettlingTime(~2%%peak) = %.3f s (require <= 5 s)\n', TsY
    );
94
95 figure;
96 plot(t_step, y1_step, 'LineWidth', 1.2); grid on
97 title('Active Suspension: y1 = x1 - x2 (STEP road, lsim)');
98 xlabel('Time (s)'); ylabel('y1 (m)');
99 yline( 0.1,'--'); yline(-0.1,'--');
100
101 figure;
102 plot(t_step, u_step, 'LineWidth', 1.2); grid on
103 title('Active Suspension: Control Force u (STEP road, lsim)');
104 xlabel('Time (s)'); ylabel('u (N)');
105
106 %% =====
107 % 5) ISO 8608 random road + RK4 simulation
108 %% =====
109 if exist('iso8608_road_Wt', 'file') ~= 2
110     error('iso8608_road_Wt.m not found on path. Put your road
        function in the same folder or addpath().');
111 end
112
113 % --- Road parameters (edit) ---
114 L      = 500;           % [m]
115 dx     = 0.02;         % [m]
116 v      = 80;           % [m/s]
117 Gd0    = 64e-6;        % [m^3] choose ISO class value
118 wavE   = 2;            % ISO exponent
119 seed   = 1;
120
121 [tRoad, WRoad] = Roadsimulation(L, dx, v, Gd0, wavE, seed);
122
123 % Continuous w(t) for RK4 midpoints
124 w_of_t = @(tt) interp1(tRoad, WRoad, tt, 'linear', 'extrap');
125
126 % Use road sampling time (recommended)
127 h      = tRoad(2) - tRoad(1);
128 tRK    = tRoad(:);
129 NRK    = numel(tRK);
130
131 xRK    = zeros(n, NRK);
132 y1RK   = zeros(NRK,1);
133 uRK    = zeros(NRK,1);
134 wRK    = zeros(NRK,1);
135
136 % initial condition
137 xRK(:,1) = zeros(n,1);
138
139 % Closed-loop dynamics: xdot = Acl*x + Bcl*w(t)

```

```

140 f = @(tt,xx) (Acl*xx + Bcl*w_of_t(tt));
141
142 for k = 1:NRK-1
143     tk = tRK(k);
144     xk = xRK(:,k);
145
146     k1 = f(tk,      xk);
147     k2 = f(tk+h/2,  xk + (h/2)*k1);
148     k3 = f(tk+h/2,  xk + (h/2)*k2);
149     k4 = f(tk+h,    xk + h*k3);
150
151     xRK(:,k+1) = xk + (h/6)*(k1 + 2*k2 + 2*k3 + k4);
152
153     wk      = w_of_t(tk);
154     wRK(k)  = wk;
155     y1RK(k) = C*xk;
156     uRK(k)  = -K*xk + Nw*wk;
157 end
158
159 % last sample
160 wRK(end) = w_of_t(tRK(end));
161 y1RK(end) = C*xRK(:,end);
162 uRK(end) = -K*xRK(:,end) + Nw*wRK(end);
163
164 % Metrics (random road)
165 peakY_RK = max(abs(y1RK));
166 rmsY_RK  = sqrt(mean(y1RK.^2));
167
168 fprintf('\n=== ISO 8608 road results (RK4) ===\n');
169 fprintf('Peak |y1| = %.4f m\n', peakY_RK);
170 fprintf('RMS  y1   = %.4f m\n', rmsY_RK);
171
172 figure;
173 plot(tRK, WRoad, 'LineWidth', 1.2); grid on
174 title('ISO 8608 Road Profile W(t)');
175 xlabel('Time (s)'); ylabel('W(t) (m)');
176
177 figure;
178 plot(tRK, y1RK, 'LineWidth', 1.2); grid on
179 title('Active Suspension: y1 response (ISO 8608 road, RK4)');
180 xlabel('Time (s)'); ylabel('y1 = x1 - x2 (m)');
181
182 figure;
183 plot(tRK, uRK, 'LineWidth', 1.2); grid on
184 title('Active Suspension: Control Force u(t) (ISO 8608 road, RK4)');
185 xlabel('Time (s)'); ylabel('u (N)');
186
187 %% =====
188 % Local helper
189 %% =====

```

```

190 function Ts = settling_time_to_zero(t, y, frac)
191 % Settling time to 0 using +/- frac*peak band
192 peak = max(abs(y));
193 band = frac*peak;
194 idx = find(abs(y) > band, 1, 'last');
195 if isempty(idx)
196     Ts = 0;
197 else
198     Ts = t(idx);
199 end
200 end

```

A.2 Passive suspension ISO 8608.m

Listing 2: Active_Suspension.m

```

1  %% Passive Suspnsion for ISO 8608 random road + RK4 simulation
2
3
4  clear; close all; clc
5
6  %% =====
7  % 0) Parameters
8  %% =====
9  m1 = 290;          % kg (sprung mass)
10 m2 = 59;           % kg (unsprung mass)
11 k1 = 16000;        % N/m (suspension spring)
12 k2 = 190000;       % N/m (tire stiffness)
13 b1 = 1000;         % N*s/m (suspension damper)
14 b2 = 0;            % N*s/m (tire damper) <-- keep 0 so road input
    is W only (no Wdot)
15
16 W_step = 0.1;      % [m] step road disturbance this is the form the
    passive suspension code
17 %% =====
18 % 1) State-space model (x = [x1; x1dot; x2; x2dot])
19 %% =====
20 A = [ 0          1          0          0;
21       -k1/m1     -b1/m1      k1/m1      b1/m1;
22       0          0          0          1;
23       k1/m2      b1/m2     -(k1+k2)/m2  -(b1+b2)/m2 ];
24
25 Bu = [0; 1/m1; 0; -1/m2]; % control force u
26 Bw = [0; 0; 0; k2/m2];    % road displacement W(t) (valid
    cleanly when b2=0)
27
28 C = [1 0 -1 0];          % y1 = x1 - x2 (suspension deflection
    )
29 Du = 0;
30
31 n = size(A,1);
32
33 %% =====
34 % 2) Controllability check (needed for place)
35 %% =====
36 Co = ctrb(A, Bu);
37 rCo = rank(Co);
38
39 fprintf('\n=== Controllability ===\n');
40 fprintf('rank(ctrb(A,Bu)) = %d, n = %d\n', rCo, n);
41
42 if rCo < n
43     error("System NOT controllable w.r.t. u: rank=%d < %d. Can't
        use place().", rCo, n);

```

```

44 end
45
46 %% =====
47 % 3) State feedback design ( $u = -Kx + Nw*W$ )
48 %% =====
49 % Choose desired closed-loop poles (edit/tune)
50 p_des = [-7.91+5.90j, -7.91-5.90j, -20.19+45.53j, -20.19-45.53j];
51
52 K = place(A, Bu, p_des);
53
54 Acl = A - Bu*K;
55
56 % Disturbance feedforward to make  $y_{ss} = 0$  for constant  $W$  (if
    possible)
57 den = C*(Acl\Bu);
58 if abs(den) < 1e-12
59     warning('C*(Acl\Bu) is ~0 -> setting Nw=0 (no road
        feedforward).');
60     Nw = 0;
61 else
62     Nw = -(C*(Acl\Bw)) / den; % scalar
63 end
64
65 Bcl = Bw + Bu*Nw;
66
67 fprintf('\n=== Controller ===\n');
68 fprintf('K = [%.6f %.6f %.6f %.6f]\n', K);
69 fprintf('Nw = %.6f\n', Nw);
70
71 %% =====
72 % 4) STEP road simulation (lsim)
73 %% =====
74 % Closed-loop from  $W \rightarrow$  outputs  $[y1; u]$ 
75 sys_cl = ss(Acl, Bcl, [C; -K], [0; Nw]);
76
77 t_step = 0:0.001:10;
78 w_vec = W_step*ones(size(t_step));
79
80 yout = lsim(sys_cl, w_vec, t_step);
81 y1_step = yout(:,1);
82 u_step = yout(:,2);
83
84 peakY = max(abs(y1_step));
85 TsY = settling_time_to_zero(t_step, y1_step, 0.02);
86
87 fprintf('\n=== STEP road results (W = 0.1 m) ===\n');
88 fprintf('Peak |y1| = %.4f m (require <= 0.1 m)\n', peakY);
89 fprintf('SettlingTime(~2%%peak) = %.3f s (require <= 5 s)\n', TsY
    );
90
91

```

```

92
93
94 %% =====
95 % 5) ISO 8608 random road + RK4 simulation
96 %% =====
97
98
99 % --- Road parameters (edit) ---
100 L      = 500;           % [m]
101 dx     = 0.02;         % [m]
102 v      = 50;           % [m/s]
103 Gd0    = 64e-6;        % [m^3] choose ISO class value
104 wavE   = 2;            % ISO exponent form the reference
105 seed   = 1;
106
107 [tRoad, WRoad] = Roadsimulation(L, dx, v, Gd0, wavE, seed);
108
109 % Continuous w(t) for RK4 midpoints
110 w_of_t = @(tt) interp1(tRoad, WRoad, tt, 'linear', 'extrap');
111
112 % Use road sampling time (recommended)
113 h      = tRoad(2) - tRoad(1);
114 tRK    = tRoad(:);
115 NRK    = numel(tRK);
116
117 xRK     = zeros(n, NRK);
118 y1RK    = zeros(NRK,1);
119 x1RK    = zeros(NRK,1);
120 x1relRK = zeros(NRK,1);
121 uRK     = zeros(NRK,1);
122 wRK     = zeros(NRK,1);
123
124 % initial condition
125 xRK(:,1) = zeros(n,1);
126
127 % Closed-loop dynamics: xdot = Acl*x + Bcl*w(t)
128 f = @(tt,xx) (Acl*xx + Bcl*w_of_t(tt));
129
130 for k = 1:NRK-1
131     tk = tRK(k);
132     xk = xRK(:,k);
133
134     k1 = f(tk, xk);
135     k2 = f(tk+h/2, xk + (h/2)*k1);
136     k3 = f(tk+h/2, xk + (h/2)*k2);
137     k4 = f(tk+h, xk + h*k3);
138
139     xRK(:,k+1) = xk + (h/6)*(k1 + 2*k2 + 2*k3 + k4);
140
141     wk = w_of_t(tk);
142     wRK(k) = wk;

```

```

143 y1RK(k)    = C*xk;
144 x1RK(k)    = xk(1);
145 x1relRK(k) = xk(1) - wk;
146 uRK(k)     = -K*xk + Nw*wk;
147
148 end
149
150 % last sample
151 wRK(end)   = w_of_t(tRK(end));
152 y1RK(end)  = C*xRK(:,end);
153 x1RK(end)  = xRK(1,end);
154 x1relRK(end) = xRK(1,end) - wRK(end);
155 uRK(end)   = -K*xRK(:,end) + Nw*wRK(end);
156
157
158 % Metrics (random road)
159 peakY_RK = max(abs(y1RK));
160 rmsY_RK  = sqrt(mean(y1RK.^2));
161
162 fprintf('\n=== ISO 8608 road results (RK4) ===\n');
163 fprintf('Peak |y1| = %.4f m\n', peakY_RK);
164 fprintf('RMS  y1   = %.4f m\n', rmsY_RK);
165
166 figure;
167 plot(tRK, WRoad, 'LineWidth', 1.2); grid on
168 title('ISO 8608 Road Profile W(t)');
169 xlabel('Time (s)'); ylabel('W(t) (m)');
170
171 figure;
172 plot(tRK, y1RK, 'LineWidth', 1.2); grid on
173 title('Active Suspension: y1 response (ISO 8608 road, RK4)');
174 xlabel('Time (s)'); ylabel('y1 = x1 - x2 (m)');
175
176 figure;
177 plot(tRK, uRK, 'LineWidth', 1.2); grid on
178 title('Active Suspension: Control Force u(t) (ISO 8608 road, RK4)');
179 xlabel('Time (s)'); ylabel('u (N)');
180
181 peakX1_RK = max(abs(x1RK));
182 rmsX1_RK  = sqrt(mean(x1RK.^2));
183
184 peakX1rel_RK = max(abs(x1relRK));
185 rmsX1rel_RK  = sqrt(mean(x1relRK.^2));
186
187 fprintf('Peak |x1|          = %.4f m, RMS x1          = %.4f m\n',
188         peakX1_RK, rmsX1_RK);
189 fprintf('Peak |x1-W|       = %.4f m, RMS x1-W       = %.4f m\n',
190         peakX1rel_RK, rmsX1rel_RK);
191
192 figure;

```

```

191 plot(tRK, x1RK, 'LineWidth', 1.2); grid on
192 title('Active Suspension: body displacement x_1 (ISO 8608 road,
193     RK4)');
194 xlabel('Time (s)'); ylabel('x_1 (m)');
195
196 figure;
197 plot(tRK, x1relRK, 'LineWidth', 1.2); grid on
198 title('Active Suspension: body relative displacement (x_1 - W) (
199     ISO 8608 road, RK4)');
200 xlabel('Time (s)'); ylabel('x_1 - W (m)');
201
202
203
204 %% =====
205 % Local helper
206 %% =====
207 function Ts = settling_time_to_zero(t, y, frac)
208 % Settling time to 0 using +/- frac*peak band
209 peak = max(abs(y));
210 band = frac*peak;
211 idx = find(abs(y) > band, 1, 'last');
212 if isempty(idx)
213     Ts = 0;
214 else
215     Ts = t(idx);
216 end
217 end

```


A.3 Roadsimulation.m

Listing 3: Roadsimulation.m

```
1 function [t,W,x,z] = iso8608_road_Wt(L, dx, v, Gd0, w, seed)
2 % ISO 8608 random road -> W(t)
3 % L [m] road length
4 % dx [m] spatial sampling step (0.025 m like profilometer
   sampling)
5 % v [m/s] vehicle speed (constant)
6 % Gd0 [m3] PSD at n0=0.1 cycles/m (choose from ISO class table)
7 % w [-] waviness exponent (ISO suggests w=2)
8 % seed RNG seed for repeatability
9
10 if nargin < 6, seed = 1; end
11 rng(seed);
12
13 % ISO reference
14 n0 = 0.1; % cycles/m
15 nmin = 0.01; % ISO typical lower bound
16 nmax = 10; % ISO typical upper bound
17 N = 3000;
18 x = (0:dx:L).';
19 n = linspace(nmin, nmax, N); % cycles/m
20 dn = n(2)-n(1);
21 Gd = Gd0 * (n./n0).-(w);
22 phi = 2*pi*rand(1,N);
23 A = sqrt(2*Gd*dn);
24 z = zeros(size(x));
25 for i = 1:N
26     z = z + A(i)*cos(2*pi*n(i)*x + phi(i));
27 end
28
29 % Conv: x = v t
30 dt = dx / v;
31 t = (0:length(x)-1).' * dt;
32
33 % Yo
34 W = z; % W(t) [m]
35
36 end
```

A.4 rampthenbump.m

Listing 4: rampthenbump.m

```

1 clear; close all; clc
2
3
4 %% =====
5 % 0) Parameters (edit these)
6 %% =====
7 % Use any "reasonable reference" values you choose for part (4)
8 m1 = 290;          % kg (sprung mass)
9 m2 = 59;          % kg (unsprung mass)
10 k1 = 16000;        % N/m (suspension spring)
11 k2 = 190000;       % N/m (tire stiffness)
12 b1 = 1000;         % N*s/m (suspension damper)
13 b2 = 0;           % N*s/m (tire damper) <-- set to 0 to avoid Wdot
    in state-space parts
14
15 W_step = 0.1;      % meters (project uses 0.1 m step road
    disturbance)
16
17 %% =====
18 % 2-3) Transfer matrix verification (Laplace)
19 %% =====
20 s = tf('s');
21
22 a11 = m1*s^2 + b1*s + k1;
23 a12 = -(b1*s + k1);
24 a21 = a12;
25 a22 = m2*s^2 + (b1+b2)*s + (k1+k2);
26
27 Delta = a11*a22 - a12*a21;
28
29 % "Given" transfer matrix in the project statement
30 H11 = (m2*s^2 + b2*s + k2)/Delta;
31 H12 = (b1*b2*s^2 + (b1*k2 + b2*k1)*s + k1*k2)/Delta;
32 H21 = (-m1*s^2)/Delta;
33 H22 = (m1*b2*s^3 + (m1*k2 + b1*b2)*s^2 + (b1*k2 + b2*k1)*s + k1*
    k2)/Delta;
34
35 H_given = [H11 H12; H21 H22];
36
37 % Compute the same matrix using inverse of Aeq(s):
38 % Aeq(s)*[X1;X2] = [U; -U+(b2*s+k2)W]
39 invA = (1/Delta)*[a22, -(a12); -(a21), a11];
40 H_u = invA*[1; -1];          % column from U
41 H_w = invA*[0; (b2*s+k2)];   % column from W
42 H_calc = [H_u(1) H_w(1); H_u(2) H_w(2)];
43
44 H_diff = minreal(H_given - H_calc);
45 disp('Part (3) check: H_given - H_calc (should be ~0):');

```

```

46 disp(H_diff);
47
48 %% =====
49 % 4) Step responses of G1 and G2 + metrics
50 %% =====
51 %  $G1 = (X1 - X2)/U$  ,  $G2 = (X1 - X2)/W$ 
52 G1 = minreal(H11 - H21);
53 G2 = minreal(H12 - H22);
54
55 t = 0:0.001:5;
56
57 %% ----- G1 : unit step in U -----
58 figure;
59 step(G1,t); grid on
60 title('G1 = (X1 - X2) / U    (Unit Step in U)')
61 ylabel('x_1 - x_2    (m)')
62 xlabel('Time (s)')
63
64 info1 = stepinfo(G1);
65 p1 = pole(G1);
66 wd1 = abs(imag(p1(find(imag(p1)>0,1))));
67
68 fprintf('\n==== G1 RESULTS =====\n');
69 fprintf('Settling Time = %.3f s\n', info1.SettlingTime);
70 fprintf('Maximum Overshoot = %.2f %%\n', info1.Overshoot);
71 fprintf('Damped Frequency wd = %.3f rad/s\n', wd1);
72
73 %% ----- G2 : 0.1 m road step -----
74 W_step = 0.1;
75 [y2,t2] = step(W_step*G2,t);
76
77 figure;
78 plot(t2,y2); grid on
79 title('G2 = (X1 - X2) / W    (0.1 m Road Step)')
80 ylabel('x_1 - x_2    (m)')
81 xlabel('Time (s)')
82
83 info2 = stepinfo(W_step*G2);      % <-- must be scaled
84
85 peak2 = max(abs(y2));
86 band = 0.02*peak2;
87 Ts2 = t2(find(abs(y2)<=band,1,'last'));
88
89 p2 = pole(G2);
90 wd2 = abs(imag(p2(find(imag(p2)>0,1))));
91
92 fprintf('\n==== G2 RESULTS =====\n');
93 fprintf('Peak Deflection = %.4f m (%.1f %% of 0.1 m)\n', peak2
94     ,100*peak2/W_step);
95 fprintf('Settling Time = %.3f s\n', Ts2);
96 fprintf('Damped Frequency wd = %.3f rad/s\n', wd2);

```

```

96
97 %% =====
98 % 5) State-space model with output  $y_1 = x_1 - x_2$ 
99 %% =====
100 % States:  $x = [x_1; \dot{x}_1; x_2; \dot{x}_2]$ 
101 A = [ 0      1      0      0;
102      -k1/m1  -b1/m1   k1/m1   b1/m1;
103      0      0      0      1;
104      k1/m2   b1/m2  -(k1+k2)/m2  -(b1+b2)/m2 ];
105
106 Bu = [0; 1/m1; 0; -1/m2];          % control force  $u$ 
107 Bw = [0; 0; 0; k2/m2];            % road displacement  $W$  (valid
    cleanly when  $b_2 = 0$ )
108
109 C = [1 0 -1 0];                    %  $y_1 = x_1 - x_2$ 
110 Du = 0; Dw = 0;
111
112 sys_uw = ss(A, [Bu Bw], C, [Du Dw]);
113
114 %% =====
115 % 6) Open-loop stability
116 %% =====
117 eigA = eig(A);
118 fprintf('\n== Part (6) Open-loop stability ==\n');
119 disp('eig(A) = '); disp(eigA.');
120 if all(real(eigA) < 0)
121     disp('Open-loop is asymptotically stable (all eigenvalues
122         have negative real parts).');
123 else
124     disp('Open-loop is NOT asymptotically stable (at least one
125         eigenvalue has Re >= 0).');
126 end
127
128 %% =====
129 % 7) Controllability and observability
130 %% =====
131 Co = ctrb(A, Bu);
132 Ob = obsv(A, C);
133
134 rCo = rank(Co);
135 rOb = rank(Ob);
136
137 fprintf('\n== Part (7) Controllability/Observability ==\n');
138 fprintf('rank(ctrb) = %d (n=%d states)\n', rCo, size(A,1));
139 fprintf('rank(obsv) = %d (n=%d states)\n', rOb, size(A,1));
140
141 %% =====
142 % 8) State feedback design for  $W = 0.1$  m step
    % Requirement:  $y_1$  stays within  $\pm 0.1$  m and oscillations vanish
    within 5 s
    %% =====

```

```

143 % Pick desired closed-loop poles (tune as needed)
144 p_des = [-7.91+5.90j, -7.91-5.90j, -20.19+45.53j, -20.19-45.53j];
145 Ks = place(A, Bu, p_des);
146
147 Acl = A - Bu*Ks;
148
149 % OPTIONAL disturbance feedforward to make y_ss = 0 for constant
    W (requires W available):
150 Nw = -(C*(Acl\Bw)) / (C*(Acl\Bu)); % scalar
151
152 % Closed-loop from W -> outputs [y1; u]
153 Bcl = Bw + Bu*Nw;
154 Caug = [C; -Ks];
155 Daug = [0; Nw];
156 sys_cl = ss(Acl, Bcl, Caug, Daug);
157
158 % Simulate W step = 0.1 m
159 t8 = 0:0.001:20;
160 w = W_step*ones(size(t8));
161 yout = lsim(sys_cl, w, t8);
162 y1 = yout(:,1);
163 u = yout(:,2);
164
165 % Check requirements
166 peakY = max(abs(y1));
167 TsY = settling_time_to_zero(t8, y1, 0.02);
168
169 fprintf('\n=== Part (8) State feedback results ===\n');
170 fprintf('Ks = [%.3f %.3f %.3f %.3f]\n', Ks);
171 fprintf('Nw (feedforward) = %.3f\n', Nw);
172 fprintf('Peak |y1| = %.4f m (require <= 0.1 m)\n', peakY);
173 fprintf('SettlingTime(~2%%peak) = %.3f s (require <= 5 s)\n', TsY
    );
174
175 figure; plot(t8, y1); grid on
176 title('Part (8): Closed-loop y1 for W = 0.1 m step'); ylabel('y1
    = x1 - x2 (m)'); xlabel('Time (s)')
177 yline( 0.1, '--'); yline(-0.1, '--');
178
179 figure; plot(t8, u); grid on
180 title('Part (8): Control force u'); ylabel('u (N)'); xlabel('Time
    (s)')
181
182
183 %% =====
184 % 8b) Road input w(t): RAMP then BUMP (after applying controller
    Ks)
185 %% =====
186 % Ramp: 0 -> W_step over Tramp seconds (starting at t_r0)
187 % Bump: half-sine pulse of height A_bump over T_bump seconds (
    starting at t_b0)

```

```

188
189 t_r0 = 0.0;      % s ramp start time
190 Tramp = 1.0;    % s ramp duration
191
192 A_bump = 0.05;  % m bump height
193 t_b0 = 3.0;    % s bump start time
194 T_bump = 2;    % s bump duration
195
196 w_rb = road_ramp_then_bump(t8, W_step, t_r0, Tramp, A_bump, t_b0,
    T_bump);
197
198 yout_rb = lsim(sys_cl, w_rb, t8);
199 y1_rb = yout_rb(:,1);
200 u_rb = yout_rb(:,2);
201
202 figure; plot(t8, w_rb); grid on
203 title('Road input w(t): ramp then bump'); ylabel('w(t) (m)');
    xlabel('Time (s)')
204
205 figure; plot(t8, y1_rb); grid on
206 title('Closed-loop y1 for w(t) = ramp then bump'); ylabel('y1 =
    x1 - x2 (m)'); xlabel('Time (s)')
207 yline( 0.1, '--'); yline(-0.1, '--');
208
209 figure; plot(t8, u_rb); grid on
210 title('Control force u for w(t) = ramp then bump'); ylabel('u (N)
    '); xlabel('Time (s)')
211
212
213 %% =====
214 % Local helper functions
215 %% =====
216 function w = road_ramp_then_bump(t, W_final, t_r0, Tramp, A_bump,
    t_b0, T_bump)
217
218 % --- Saturating ramp ---
219 if Tramp <= 0
220     r = double(t >= t_r0);          % instantaneous step if
        Tramp is invalid
221 else
222     r = (t - t_r0) / Tramp;
223     r = max(0, min(r, 1));          % clamp to [0,1]
224 end
225 w = W_final * r;
226
227 % --- Half-sine bump ---
228 tau = t - t_b0;
229 w = w + A_bump * (tau >= 0 & tau <= T_bump) .* sin(pi * tau /
    T_bump);
230 end
231

```

```

232 function [pdom, wd] = dominant_complex_pole(p)
233 % Pick the complex pole closest to the imaginary axis (dominant
    oscillatory mode)
234 pc = p(imag(p) ~= 0);
235 if isempty(pc)
236     pdom = NaN; wd = NaN; return;
237 end
238 [~, idx] = min(abs(real(pc)));
239 pdom = pc(idx);
240 wd = abs(imag(pdom)); % rad/s
241 end
242
243 function Ts = settling_time_to_zero(t, y, frac)
244 % Settling time to 0 using +/- frac*peak band, where peak = max(|
    y|)
245 peak = max(abs(y));
246 band = frac*peak;
247 idx = find(abs(y) > band, 1, 'last');
248 if isempty(idx), Ts = 0; else, Ts = t(idx); end
249 end

```

A.5 Newiteration.m

Listing 5: Passive Suspension.m

```

1  clear; close all; clc
2
3
4  %% =====
5  % 0) Parameters (edit these)
6  %% =====
7  % Use any "reasonable reference" values you choose for part (4)
8  m1 = 290;          % kg (sprung mass)
9  m2 = 59;          % kg (unsprung mass)
10 k1 = 16000;        % N/m (suspension spring)
11 k2 = 190000;       % N/m (tire stiffness)
12 b1 = 1000;        % N*s/m (suspension damper)
13 b2 = 0;           % N*s/m (tire damper)  <-- set to 0 to avoid Wdot
    in state-space parts
14
15 W_step = 0.1;      % meters (project uses 0.1 m step road
    disturbance) :contentReference[oaicite:1]{index=1}
16
17 %% =====
18 % 2-3) Transfer matrix verification (Laplace)
19 %% =====
20 s = tf('s');
21
22 a11 = m1*s^2 + b1*s + k1;
23 a12 = -(b1*s + k1);
24 a21 = a12;
25 a22 = m2*s^2 + (b1+b2)*s + (k1+k2);
26
27 Delta = a11*a22 - a12*a21;
28
29 % "Given" transfer matrix in the project statement :
    contentReference[oaicite:2]{index=2}
30 H11 = (m2*s^2 + b2*s + k2)/Delta;
31 H12 = (b1*b2*s^2 + (b1*k2 + b2*k1)*s + k1*k2)/Delta;
32 H21 = (-m1*s^2)/Delta;
33 H22 = (m1*b2*s^3 + (m1*k2 + b1*b2)*s^2 + (b1*k2 + b2*k1)*s + k1*
    k2)/Delta;
34
35 H_given = [H11 H12; H21 H22];
36
37 % Compute the same matrix using inverse of Aeq(s):
38 % Aeq(s)*[X1;X2] = [U; -U+(b2*s+k2)W]
39 invA = (1/Delta)*[a22, -(a12); -(a21), a11];
40 H_u = invA*[1; -1];          % column from U
41 H_w = invA*[0; (b2*s+k2)];   % column from W
42 H_calc = [H_u(1) H_w(1); H_u(2) H_w(2)];
43
44 H_diff = minreal(H_given - H_calc);

```



```

45 disp('Part (3) check: H_given - H_calc (should be ~0:');
46 disp(H_diff);
47
48 %% =====
49 % 4) Step responses of G1 and G2 + metrics
50 %% =====
51 %  $G1 = (X1 - X2)/U$  ,  $G2 = (X1 - X2)/W$  :contentReference[oaicite
    :3]{index=3}
52 G1 = minreal(H11 - H21);
53 G2 = minreal(H12 - H22);
54
55 t = 0:0.001:5;
56
57 %% ----- G1 : unit step in U -----
58 figure;
59 step(G1,t); grid on
60 title('G1 = (X1 - X2) / U    (Unit Step in U)')
61 ylabel('x_1 - x_2    (m)')
62 xlabel('Time (s)')
63
64 info1 = stepinfo(G1);
65 p1 = pole(G1);
66 wd1 = abs(imag(p1(find(imag(p1)>0,1))));
67
68 fprintf('\n==== G1 RESULTS =====\n');
69 fprintf('Settling Time = %.3f s\n', info1.SettlingTime);
70 fprintf('Maximum Overshoot = %.2f %%\n', info1.Overshoot);
71 fprintf('Damped Frequency wd = %.3f rad/s\n', wd1);
72
73
74 %% ----- G2 : 0.1 m road step -----
75
76 W_step = 0.1;
77 [y2,t2] = step(W_step*G2,t);
78
79 figure;
80 plot(t2,y2); grid on
81 title('G2 = (X1 - X2) / W    (0.1 m Road Step)')
82 ylabel('x_1 - x_2    (m)')
83 xlabel('Time (s)')
84
85 info2 = stepinfo(W_step*G2);          % <-- must be scaled
86
87 peak2 = max(abs(y2));
88 band = 0.02*peak2;
89 Ts2 = t2(find(abs(y2)<=band,1,'last'));
90
91 p2 = pole(G2);
92 wd2 = abs(imag(p2(find(imag(p2)>0,1))));
93
94 fprintf('\n==== G2 RESULTS =====\n');

```

```

95 fprintf('Peak Deflection = %.4f m (%.1f %% of 0.1 m)\n', peak2
    ,100*peak2/W_step);
96 fprintf('Settling Time = %.3f s\n', Ts2);
97 fprintf('Damped Frequency wd = %.3f rad/s\n', wd2);
98
99 %% =====
100 % 5) State-space model with output y1 = x1 - x2
101 %% =====
102 % States: x = [x1; x1dot; x2; x2dot]
103 A = [ 0          1          0          0;
104       -k1/m1     -b1/m1      k1/m1      b1/m1;
105       0          0          0          1;
106       k1/m2      b1/m2     -(k1+k2)/m2  -(b1+b2)/m2 ];
107
108 Bu = [0; 1/m1; 0; -1/m2];           % control force u
109 Bw = [0; 0; 0; k2/m2];             % road displacement W (valid
    cleanly when b2 = 0)
110
111 C = [1 0 -1 0];                    % y1 = x1 - x2
112 Du = 0; Dw = 0;
113
114 sys_uw = ss(A, [Bu Bw], C, [Du Dw]);
115
116 %% =====
117 % 6) Open-loop stability
118 %% =====
119 eigA = eig(A);
120 fprintf('\n=== Part (6) Open-loop stability ===\n');
121 disp('eig(A) = '); disp(eigA.');
122 if all(real(eigA) < 0)
123     disp('Open-loop is asymptotically stable (all eigenvalues
        have negative real parts).');
124 else
125     disp('Open-loop is NOT asymptotically stable (at least one
        eigenvalue has Re >= 0).');
126 end
127
128 %% =====
129 % 7) Controllability and observability
130 %% =====
131 Co = ctrb(A, Bu);
132 Ob = obsv(A, C);
133
134 rCo = rank(Co);
135 rOb = rank(Ob);
136
137 fprintf('\n=== Part (7) Controllability/Observability ===\n');
138 fprintf('rank(ctrb) = %d (n=%d states)\n', rCo, size(A,1));
139 fprintf('rank(observ) = %d (n=%d states)\n', rOb, size(A,1));
140
141 %% =====

```

```

142 % 8) State feedback design for W = 0.1 m step
143 % Requirement: y1 stays within +/-0.1 m and oscillations vanish
      within 5 s :contentReference[oaicite:4]{index=4}
144 %% =====
145 % Pick desired closed-loop poles (tune as needed)
146 p_des = [-7.91+5.90j, -7.91-5.90j, -20.19+45.53j, -20.19-45.53j];
147 K = place(A, Bu, p_des);
148
149 Acl = A - Bu*K;
150
151 % OPTIONAL disturbance feedforward to make y_ss = 0 for constant
      W (requires W available):
152 Nw = -(C*(Acl\Bw)) / (C*(Acl\Bu)); % scalar
153
154 % Closed-loop from W -> outputs [y1; u]
155 Bcl = Bw + Bu*Nw;
156 Caug = [C; -K];
157 Daug = [0; Nw];
158 sys_cl = ss(Acl, Bcl, Caug, Daug);
159
160 % Simulate W step = 0.1 m
161 t8 = 0:0.001:10;
162 w = W_step*ones(size(t8));
163 yout = lsim(sys_cl, w, t8);
164 y1 = yout(:,1);
165 u = yout(:,2);
166
167 % Check requirements
168 peakY = max(abs(y1));
169 TsY = settling_time_to_zero(t8, y1, 0.02);
170
171 fprintf('\n== Part (8) State feedback results ==\n');
172 fprintf('K = [%3f %3f %3f %3f]\n', K);
173 fprintf('Nw (feedforward) = %3f\n', Nw);
174 fprintf('Peak |y1| = %4f m (require <= 0.1 m)\n', peakY);
175 fprintf('SettlingTime(~2%%peak) = %3f s (require <= 5 s)\n', TsY
      );
176
177 figure; plot(t8, y1); grid on
178 title('Part (8): Closed-loop y1 for W = 0.1 m step'); ylabel('y1
      = x1 - x2 (m)'); xlabel('Time (s)')
179 yline( 0.1,'--'); yline(-0.1,'--');
180
181 figure; plot(t8, u); grid on
182 title('Part (8): Control force u'); ylabel('u (N)'); xlabel('Time
      (s)')
183
184 %% =====
185 % Local helper functions
186 %% =====
187 function [pdom, wd] = dominant_complex_pole(p)

```

```

188 % Pick the complex pole closest to the imaginary axis (dominant
      oscillatory mode)
189 pc = p(imag(p) ~= 0);
190 if isempty(pc)
191     pdom = NaN; wd = NaN; return;
192 end
193 [~, idx] = min(abs(real(pc)));
194 pdom = pc(idx);
195 wd = abs(imag(pdom)); % rad/s
196 end
197
198 function Ts = settling_time_to_zero(t, y, frac)
199 % Settling time to 0 using +/- frac*peak band, where peak = max(|
      y|)
200 peak = max(abs(y));
201 band = frac*peak;
202 idx = find(abs(y) > band, 1, 'last');
203 if isempty(idx), Ts = 0; else, Ts = t(idx); end
204 end

```

A.6 newii.m

Listing 6: Passive Simulink code.m

```

1 % Parameters
2 m1 = 290; m2 = 59;
3 k1 = 16000; k2 = 190000;
4 b1 = 1000; b2 = 0; % keep b2=0 to avoid needing Wdot
5
6 % State-space (x=[x1 x1dot x2 x2dot]')
7 A = [ 0 1 0 0;
8       -k1/m1 -b1/m1 k1/m1 b1/m1;
9       0 0 0 1;
10      k1/m2 b1/m2 -(k1+k2)/m2 -(b1+b2)/m2 ];
11
12 Bu = [0; 1/m1; 0; -1/m2];
13 Bw = [0; 0; 0; k2/m2];
14
15 C_y1 = [1 0 -1 0]; % y1 = x1 - x2
16 D_y1 = [0 0];
17
18 % Part (8) desired poles (example)
19 p_des = [-7.91+5.90j, -7.91-5.90j, -20.19+45.53j, -20.19-45.53j];
20
21 % State feedback gain
22 K = place(A, Bu, p_des);
23
24 % Feedforward so steady-state y1 -> 0 for a step in W (important)
25 Ac1 = A - Bu*K;
26 Nw = -(C_y1*(Ac1\Bw)) / (C_y1*(Ac1\Bu)); % scalar

```

A.7 lqr_pcode.m

```

1 function lqr_pcode %control terrain type by modifying line 47
2 clear; clc; close all;
3
4 m1 = 250.0; m2 = 45.0;
5 k1 = 16000.0; k2 = 160000.0;
6 b1 = 1000.0; b2 = 0.0;
7
8 A = [0, 1, 0, 0;
9       -k1/m1, -b1/m1, k1/m1, b1/m1;
10      0, 0, 0, 1;
11      k1/m2, b1/m2, -(k1+k2)/m2, -(b1+b2)/m2];
12 Bu = [0; 1/m1; 0; -1/m2];
13 Bw = [0; 0; 0; k2/m2];
14
15 dt = 0.002;
16 T_end = 20;
17 t = 0:dt:T_end;
18

```

```

19 sys_pass_c = ss(A, [Bu, Bw], eye(4), 0);
20 sys_pass_d = c2d(sys_pass_c, dt, 'zoh');
21
22 Q_step = diag([40000, 100, 10, 10]);
23 R_step = 0.0003;
24 [K_step, ~, ~] = lqr(A, Bu, Q_step, R_step);
25
26 sys_active_d_step = c2d(ss(A - Bu*K_step, Bw, eye(4), 0), dt,
    'zoh');
27 road_step = 0.1 * (t >= 2.0);
28
29 [y_p_step, ~] = lsim(sys_pass_d, [zeros(size(t)); road_step
    ], t);
30 [y_a_step, ~, x_a_step] = lsim(sys_active_d_step, road_step,
    t);
31
32 force_step = (-K_step * x_a_step)';
33 deflect_p_step = y_p_step(:,1) - y_p_step(:,3);
34 deflect_a_step = y_a_step(:,1) - y_a_step(:,3);
35
36 Q_mix = diag([500, 1e8, 100, 1000]);
37 R_mix = 1e-6;
38 [K_mix, ~, ~] = lqr(A, Bu, Q_mix, R_mix);
39
40 sys_active_d_mix = c2d(ss(A - Bu*K_mix, Bw, eye(4), 0), dt, '
    zoh');
41
42 freq = 1.27;
43 fade = min(t / 3.0, 1);
44 w_comp = (0.1 * sin(2*pi*freq*t)) .* fade;
45 bumps = 0.15 * exp(-((t - 5).^2)/0.08) + 0.12 * exp(-((t -
    12).^2)/0.32);
46 rng(100); n_comp = 0.005 * randn(size(t));
47 road_mix = w_comp + bumps + n_comp; %control terrain shape
    from here.
48
49 [y_p_mix, ~] = lsim(sys_pass_d, [zeros(size(t)); road_mix]',
    t);
50 [y_a_mix, ~, x_a_mix] = lsim(sys_active_d_mix, road_mix, t);
51
52 force_mix = (-K_mix * x_a_mix)';
53 deflect_p_mix = y_p_mix(:,1) - y_p_mix(:,3);
54 deflect_a_mix = y_a_mix(:,1) - y_a_mix(:,3);
55
56 figure('Name', 'Body Displacement', 'Color', 'w', 'Position',
    [50, 50, 600, 800]);
57 subplot(2,1,1); hold on; grid on;
58 plot(t, road_step, 'k'); plot(t, y_p_step(:,1), 'r--'); plot(
    t, y_a_step(:,1), 'b', 'LineWidth', 2);
59 title('Step: Body Displacement (x1)'); legend('Road', '
    Passive', 'Active');

```

```

60 subplot(2,1,2); hold on; grid on;
61 plot(t, road_mix, 'k'); plot(t, y_p_mix(:,1), 'r--'); plot(t,
62     y_a_mix(:,1), 'b', 'LineWidth', 2);
63 title('Mixed: Body Displacement (x1)');
64
65 figure('Name', 'Control Force', 'Color', 'w', 'Position',
66     [660, 50, 600, 800]);
67 subplot(2,1,1); plot(t, force_step, 'm', 'LineWidth', 1.5);
68     grid on;
69 title('Step: Control Force u(t)'); ylabel('Force (N)'); xlim
70     ([1.5 4]);
71
72 subplot(2,1,2); plot(t, force_mix, 'm', 'LineWidth', 1.5);
73     grid on;
74 title('Mixed: Control Force u(t)'); ylabel('Force (N)');
75
76 figure('Name', 'Suspension Deflection', 'Color', 'w', '
77     Position', [1270, 50, 600, 800]);
78 subplot(2,1,1); hold on; grid on;
79 plot(t, deflect_p_step, 'r--'); plot(t, deflect_a_step, 'b',
80     'LineWidth', 2);
81 title('Step: Suspension Deflection (x1 - x3)'); ylabel('
82     Deflection (m)'); legend('Passive', 'Active');
83
84 subplot(2,1,2); hold on; grid on;
85 plot(t, deflect_p_mix, 'r--'); plot(t, deflect_a_mix, 'b', '
86     LineWidth', 2);
87 title('Mixed: Suspension Deflection (x1 - x3)'); ylabel('
88     Deflection (m)');
89 end

```

A.8 animation_gui

```

1 function animation_gui
2 clear; clc; close all;
3
4 m1 = 250.0; m2 = 45.0;
5 k1 = 16000.0; k2 = 160000.0;
6 b1 = 1000.0; b2 = 0.0;
7 velocity = 12.0;
8
9 A = [0, 1, 0, 0;
10     -k1/m1, -b1/m1, k1/m1, b1/m1;
11     0, 0, 0, 1;
12     k1/m2, b1/m2, -(k1+k2)/m2, -(b1+b2)/m2];
13 Bu = [0; 1/m1; 0; -1/m2];
14 Bw = [0; 0; 0; k2/m2];
15 B_sim = [Bu, Bw];
16

```

```

17 dt = 0.002;
18 T_end = 20;
19 t = 0:dt:T_end;
20 sys_pass_c = ss(A, B_sim, eye(4), 0);
21 sys_pass_d = c2d(sys_pass_c, dt, 'zoh');
22
23 road_profile = zeros(size(t));
24 x1_p = zeros(size(t)); x2_p = zeros(size(t));
25 x1_a = zeros(size(t)); x2_a = zeros(size(t));
26
27 is_playing = false;
28 current_idx = 1;
29
30 f_main = figure('Name', 'Active Suspension Simulator', ...
31     'Color', [0.95 0.95 0.95], 'Position', [100, 100, 1100,
32     600], ...
33     'NumberTitle', 'off', 'MenuBar', 'none');
34 pnl_ctrl = uipanel(f_main, 'Title', 'Controls', 'FontSize',
35     12, ...
36     'Position', [0.01 0.01 0.2 0.98], 'BackgroundColor', 'w')
37     ;
38 uicontrol(pnl_ctrl, 'Style', 'text', 'String', 'Terrain Types
39     :', ...
40     'Position', [10 520 180 20], 'HorizontalAlignment', 'left
41     ', ...
42     'FontSize', 10, 'FontWeight', 'bold', 'BackgroundColor',
43     'w');
44
45 chk_waves = uicontrol(pnl_ctrl, 'Style', 'checkbox', 'String'
46     , 'Resonant Waves', ...
47     'Position', [20 490 180 20], 'Value', 1, 'FontSize', 10,
48     ...
49     'BackgroundColor', 'w', 'Callback', @update_simulation);
50
51 chk_bumps = uicontrol(pnl_ctrl, 'Style', 'checkbox', 'String'
52     , 'Speedbumps', ...
53     'Position', [20 460 180 20], 'Value', 0, 'FontSize', 10,
54     ...
55     'BackgroundColor', 'w', 'Callback', @update_simulation);
56
57 chk_noise = uicontrol(pnl_ctrl, 'Style', 'checkbox', 'String'
58     , 'Gravel (Noise)', ...
59     'Position', [20 430 180 20], 'Value', 0, 'FontSize', 10,
60     ...
61     'BackgroundColor', 'w', 'Callback', @update_simulation);
62
63 chk_step = uicontrol(pnl_ctrl, 'Style', 'checkbox', 'String',
64     'Step Input (Curb)', ...
65     'Position', [20 400 180 20], 'Value', 0, 'FontSize', 10,
66     ...

```



```

54         'FontWeight', 'bold', 'ForegroundColor', 'b', ...
55         'BackgroundColor', 'w', 'Callback', @update_simulation);
56
57 btn_play = uicontrol(pnl_ctrl, 'Style', 'pushbutton', 'String
58     ', 'PLAY', ...
59     'Position', [20 100 160 50], 'FontSize', 14, 'FontWeight'
60     , 'bold', ...
61     'BackgroundColor', [0.2 0.8 0.2], 'Callback',
62     @toggle_play);
63
64 btn_reset = uicontrol(pnl_ctrl, 'Style', 'pushbutton', '
65     String', 'RESET', ...
66     'Position', [20 40 160 40], 'FontSize', 12, ...
67     'BackgroundColor', [0.8 0.8 0.8], 'Callback', @reset_sim)
68     ;
69
70 ax_p = axes('Parent', f_main, 'Position', [0.25 0.55 0.35
71     0.4]);
72 title(ax_p, 'Passive Suspension', 'Color', 'r', 'FontSize',
73     14);
74 axis(ax_p, 'equal'); xlim(ax_p, [-2 4]); ylim(ax_p, [-0.5
75     2.5]); axis(ax_p, 'off');
76 hold(ax_p, 'on');
77
78 ax_a = axes('Parent', f_main, 'Position', [0.63 0.55 0.35
79     0.4]);
80 title(ax_a, 'Active LQR Suspension', 'Color', 'b', 'FontSize'
81     , 14);
82 axis(ax_a, 'equal'); xlim(ax_a, [-2 4]); ylim(ax_a, [-0.5
83     2.5]); axis(ax_a, 'off');
84 hold(ax_a, 'on');
85
86 ax_g = axes('Parent', f_main, 'Position', [0.25 0.08 0.73
87     0.35]);
88 grid(ax_g, 'on'); hold(ax_g, 'on');
89 title(ax_g, 'Displacement Tracking', 'FontSize', 11);
90 xlabel(ax_g, 'Time (s)'); ylabel(ax_g, 'Amp (m)');
91 xlim(ax_g, [0 T_end]); ylim(ax_g, [-0.3 0.5]);
92
93 trace_road = plot(ax_g, 0, 0, 'k-', 'LineWidth', 1, '
94     DisplayName', 'Road');
95 trace_pass = plot(ax_g, 0, 0, 'r--', 'LineWidth', 1.5, '
96     DisplayName', 'Passive');
97 trace_act = plot(ax_g, 0, 0, 'b-', 'LineWidth', 2, '
98     DisplayName', 'Active');
99 legend(ax_g, 'Location', 'northeast');
100
101 road_vis_p = plot(ax_p, 0,0, 'k-', 'LineWidth', 2);
102 body_vis_p = rectangle(ax_p, 'Position', [-0.4 0.8 0.8 0.3],
103     'FaceColor', 'r', 'EdgeColor', 'k');

```

```

88 wheel_vis_p = rectangle(ax_p, 'Position', [-0.15 -0.15 0.3
      0.3], 'Curvature', [1 1], 'FaceColor', 'k');
89 spring_vis_p = plot(ax_p, [0 0], [0 0], 'Color', [0.5 0.5
      0.5], 'LineWidth', 3);
90
91 road_vis_a = plot(ax_a, 0,0, 'k-', 'LineWidth', 2);
92 body_vis_a = rectangle(ax_a, 'Position', [-0.4 0.8 0.8 0.3],
      'FaceColor', 'b', 'EdgeColor', 'k');
93 wheel_vis_a = rectangle(ax_a, 'Position', [-0.15 -0.15 0.3
      0.3], 'Curvature', [1 1], 'FaceColor', 'k');
94 spring_vis_a = plot(ax_a, [0 0], [0 0], 'Color', [0.5 0.5
      0.5], 'LineWidth', 3);
95
96 road_x_vis = linspace(-2, 4, 100);
97
98 update_simulation();
99
100 function update_simulation(~, ~)
101     if chk_step.Value == 1
102         set(chk_waves, 'Value', 0, 'Enable', 'off');
103         set(chk_bumps, 'Value', 0, 'Enable', 'off');
104         set(chk_noise, 'Value', 0, 'Enable', 'off');
105
106         Q_gain = diag([40000, 100, 10, 10]);
107         R_gain = 0.0003;
108     else
109         set(chk_waves, 'Enable', 'on');
110         set(chk_bumps, 'Enable', 'on');
111         set(chk_noise, 'Enable', 'on');
112
113         Q_gain = diag([500, 1e8, 100, 1000]);
114         R_gain = 1e-6;
115     end
116
117     [K_lqr, ~, ~] = lqr(A, Bu, Q_gain, R_gain);
118
119     sys_cont = ss(A - Bu*K_lqr, Bw, eye(4), 0);
120     sys_disc = c2d(sys_cont, dt, 'zoh');
121
122     use_wave = chk_waves.Value;
123     use_bump = chk_bumps.Value;
124     use_noise = chk_noise.Value;
125     use_step = chk_step.Value;
126
127     freq_res = 1.27;
128     fade_env = min(t / 3.0, 1);
129
130     w_comp = use_wave * (0.1 * sin(2*pi*freq_res*t)) .*
      fade_env;
131
132     bump1 = 0.15 * exp(-((t - 5.0).^2) / (2*0.2^2));

```

```

133     bump2 = 0.12 * exp(-((t - 12.0).^2) / (2*0.4^2));
134     b_comp = use_bump * (bump1 + bump2);
135
136     rng(100);
137     n_comp = use_noise * (0.015 * randn(size(t)));
138
139     s_comp = use_step * (0.1 * (t >= 2.0));
140
141     road_profile = w_comp + b_comp + n_comp + s_comp;
142
143     U_pass = [zeros(size(t)); road_profile]';
144     [y_p, ~] = lsim(sys_pass_d, U_pass, t);
145     x1_p = y_p(:,1); x2_p = y_p(:,3);
146
147     [y_a, ~] = lsim(sys_disc, road_profile, t);
148     x1_a = y_a(:,1); x2_a = y_a(:,3);
149
150     set(trace_road, 'XData', t, 'YData', road_profile);
151
152     if ~is_playing
153         draw_frame(current_idx);
154     end
155 end
156 function toggle_play(~, ~)
157     if is_playing
158         is_playing = false;
159         btn_play.String = 'PLAY';
160         btn_play.BackgroundColor = [0.2 0.8 0.2];
161     else
162         is_playing = true;
163         btn_play.String = 'PAUSE';
164         btn_play.BackgroundColor = [0.9 0.6 0.2];
165         run_animation_loop();
166     end
167 end
168 function reset_sim(~, ~)
169     is_playing = false;
170     btn_play.String = 'PLAY';
171     btn_play.BackgroundColor = [0.2 0.8 0.2];
172     current_idx = 1;
173     draw_frame(1);
174 end
175 function run_animation_loop()
176     skip_frames = 1;
177     while is_playing && current_idx < length(t)
178         current_idx = current_idx + skip_frames;
179         if current_idx > length(t)
180             current_idx = length(t);
181             is_playing = false;
182             btn_play.String = 'PLAY';
183         end

```

```

184         draw_frame(current_idx);
185         drawnow limitrate;
186     end
187 end
188 function draw_frame(idx)
189     curr_t = t(idx);
190     set(trace_pass, 'XData', t(1:idx), 'YData', x1_p(1:idx));
191     set(trace_act, 'XData', t(1:idx), 'YData', x1_a(1:idx));
192
193     road_y_vis = interp1(t, road_profile, curr_t + road_x_vis
194         /velocity, 'linear', 0);
195
196     set(road_vis_p, 'XData', road_x_vis, 'YData', road_y_vis)
197     ;
198     pos_w_p = x2_p(idx) + 0.15;
199     pos_b_p = x1_p(idx) + 0.65;
200     set(wheel_vis_p, 'Position', [-0.15, pos_w_p-0.15, 0.3,
201         0.3]);
202     set(body_vis_p, 'Position', [-0.4, pos_b_p, 0.8, 0.3]);
203     set(spring_vis_p, 'XData', [0 0], 'YData', [pos_w_p,
204         pos_b_p]);
205
206     set(road_vis_a, 'XData', road_x_vis, 'YData', road_y_vis)
207     ;
208     pos_w_a = x2_a(idx) + 0.15;
209     pos_b_a = x1_a(idx) + 0.65;
210     set(wheel_vis_a, 'Position', [-0.15, pos_w_a-0.15, 0.3,
211         0.3]);
212     set(body_vis_a, 'Position', [-0.4, pos_b_a, 0.8, 0.3]);
213     set(spring_vis_a, 'XData', [0 0], 'YData', [pos_w_a,
214         pos_b_a]);
215 end
216 end

```

A.9 LQR python comfort analysis animation (animation.py)

```

1  import numpy as np
2  import control as ct
3  import scipy.signal as signal
4  import scipy.ndimage as ndimage
5  import matplotlib.pyplot as plt
6  import matplotlib.animation as animation
7  from matplotlib.patches import Rectangle, Circle
8  from matplotlib.widgets import CheckButtons
9
10 m1, m2 = 250.0, 45.0
11 k1, k2 = 16000.0, 160000.0
12 b1, b2 = 1000.0, 0.0
13
14 A = np.array([

```

```

15     [0, 1, 0, 0],
16     [-k1/m1, -b1/m1, k1/m1, b1/m1],
17     [0, 0, 0, 1],
18     [k1/m2, b1/m2, -(k1+k2)/m2, -(b1+b2)/m2]
19 ])
20 B_u = np.array([[0], [1/m1], [0], [-1/m2]])
21 B_w = np.array([[0], [0], [0], [k2/m2]])
22
23 sys_pass = ct.ss(A, B_w, np.eye(4), np.zeros((4,1)))
24
25 q_body_pos = 500.0
26 q_body_vel = 100000.0
27 q_whl_pos = 100.0
28 q_whl_vel = 10.0
29 r_force = 0.0001
30
31 Q = np.diag([q_body_pos, q_body_vel, q_whl_pos, q_whl_vel])
32 R = r_force
33
34 K_lqr, _, _ = ct.lqr(A, B_u, Q, R)
35 sys_cl = ct.ss(A - B_u @ K_lqr, B_w, np.eye(4), np.zeros((4,1)))
36
37 T_max = 20.0
38 dt = 0.001
39 T = np.arange(0, T_max, dt)
40 velocity = 12.0
41
42 np.random.seed(60)
43 ROUGHNESS_SCALE = 0.1
44 white_noise = np.random.normal(0, 2.0, len(T))
45 b_filter, a_filter = signal.butter(1, 3.0, btype='low', analog=
    False, fs=1/dt)
46 raw_roughness = signal.lfilter(b_filter, a_filter, white_noise) *
    ROUGHNESS_SCALE
47
48 raw_bumps = np.zeros_like(T)
49 num_bumps = 6
50 bump_times = np.sort(np.random.uniform(2.0, T_max - 2.0,
    num_bumps))
51 for t_bump in bump_times:
52     h = 0.15
53     width_spatial = 0.4
54     sigma_t = (width_spatial / velocity) / (2 * np.sqrt(2 * np.
        log(2)))
55     raw_bumps += h * np.exp(-0.5 * ((T - t_bump) / sigma_t) ** 2)
56
57 freq_resonance = 1.27
58 raw_resonance = 0.1 * np.sin(2 * np.pi * freq_resonance * T)
59 raw_resonance[T < 1.0] = 0
60
61 W = np.zeros_like(T)

```

```

62 x1_p, x2_p = np.zeros_like(T), np.zeros_like(T)
63 x1_a, x2_a = np.zeros_like(T), np.zeros_like(T)
64 rms_p, rms_a = np.zeros_like(T), np.zeros_like(T)
65
66 def calculate_rolling_rms(signal_data, window_size):
67     return np.sqrt(ndimage.uniform_filter1d(signal_data**2, size=
        window_size))
68
69 def run_simulation(use_rough, use_bumps, use_resonance):
70     global W, x1_p, x2_p, x1_a, x2_a, rms_p, rms_a
71
72     W[:] = 0
73     if use_resonance:
74         W = raw_resonance.copy()
75     else:
76         if use_rough: W += raw_roughness
77         if use_bumps: W += raw_bumps
78         W[T < 0.5] = 0
79
80     _, x_pass = ct.forced_response(sys_pass, T, W)
81     _, x_act = ct.forced_response(sys_cl, T, W)
82
83     x1_p, x2_p = x_pass[0], x_pass[2]
84     x1_a, x2_a = x_act[0], x_act[2]
85
86     vert_acc_p = np.gradient(x_pass[1], dt)
87     vert_acc_a = np.gradient(x_act[1], dt)
88
89     window = int(1.0 / dt)
90     rms_p = calculate_rolling_rms(vert_acc_p, window)
91     rms_a = calculate_rolling_rms(vert_acc_a, window)
92
93     try:
94         if use_resonance:
95             ax_pass.set_title("Passive (RESONANCE FAILURE!)",
                fontsize=12, color='red', fontweight='bold')
96         else:
97             ax_pass.set_title("Passive Suspension", fontsize=12,
                color='red', fontweight='bold')
98     except NameError:
99         pass
100
101 fig1 = plt.figure(figsize=(10, 8))
102 fig1.canvas.manager.set_window_title("Main Simulation")
103 gs1 = fig1.add_gridspec(2, 2, height_ratios=[2, 1])
104
105 ax_pass = fig1.add_subplot(gs1[0, 0])
106 ax_pass.set_title("Passive Suspension", fontsize=12, color='red',
    fontweight='bold')
107 ax_pass.set_xlim(-2, 4); ax_pass.set_ylim(-0.3, 2.3); ax_pass.
    set_aspect('equal'); ax_pass.axis('off')

```

```

108
109 ax_act = fig1.add_subplot(gs1[0, 1])
110 ax_act.set_title("Active LQR", fontsize=12, color='blue',
111                 fontweight='bold')
112 ax_act.set_xlim(-2, 4); ax_act.set_ylim(-0.3, 2.3); ax_act.
113     set_aspect('equal'); ax_act.axis('off')
114
115 ax_plot = fig1.add_subplot(gs1[1, :])
116 ax_plot.set_xlim(0, T_max); ax_plot.set_ylim(-0.2, 0.4)
117 ax_plot.set_xlabel("Time (s)"); ax_plot.set_ylabel("Road Profile
118 (m)")
119 ax_plot.grid(True)
120
121 line_road, = ax_plot.plot([], [], 'k-', linewidth=1.5, alpha=0.6,
122     label='Terrain')
123 line_pass, = ax_plot.plot([], [], 'r--', linewidth=1.5, label='
124     Passive Disp.')
125 line_act, = ax_plot.plot([], [], 'b-', linewidth=2, label='Active
126     Disp.')
127 ax_plot.legend(loc='upper right')
128
129 run_simulation(True, True, False)
130
131 rax = plt.axes([0.02, 0.82, 0.15, 0.14])
132 check = CheckButtons(rax, ('Roughness', 'Bumps', 'Resonant Road')
133     , (True, True, False))
134
135 def toggle_callback(label):
136     status = check.get_status()
137     run_simulation(status[0], status[1], status[2])
138     fig1.canvas.draw_idle()
139
140 check.on_clicked(toggle_callback)
141
142 fig2 = plt.figure(figsize=(9, 5))
143 fig2.canvas.manager.set_window_title("Comfort Analysis")
144 ax_rms = fig2.add_subplot(111)
145 ax_rms.set_title("Vertical Vibration Intensity (RMS)", fontsize
146     =11, fontweight='bold')
147 ax_rms.set_xlim(0, T_max); ax_rms.set_ylim(0, 12.0)
148 ax_rms.set_xlabel("Time (s)"); ax_rms.set_ylabel("Vertical Accel
149     RMS (m/s )")
150 ax_rms.grid(True, alpha=0.3)
151
152 line_rms_p, = ax_rms.plot([], [], 'r--', linewidth=1.5, alpha
153     =0.6, label='Passive')
154 line_rms_a, = ax_rms.plot([], [], 'b-', linewidth=2, label='
155     Active')
156 text_score = ax_rms.text(0.02, 0.9, "", transform=ax_rms.
157     transAxes, fontsize=12, fontweight='bold', color='green')
158 ax_rms.legend(loc='upper right')

```

```

147 fig2.tight_layout()
148
149 def create_car_artists(ax, color):
150     road, = ax.plot([], [], 'k-', linewidth=2)
151     wheel = Circle((0, 0), 0.15, color='black', zorder=10)
152     body = Rectangle((0, 0), 0.8, 0.3, color=color, ec='black',
153                     alpha=0.8, zorder=5)
154     spring, = ax.plot([], [], color='gray', linewidth=3,
155                       linestyle='--')
156     ref_line, = ax.plot([], [], color='black', linewidth=0.5,
157                          linestyle=':')
158     deflection_line, = ax.plot([], [], color='orange', linewidth
159                                =2, zorder=20)
160     label = ax.text(0, 0, "Body Deflection", ha='center',
161                     fontsize=8)
162     ax.add_patch(wheel); ax.add_patch(body)
163     return road, wheel, body, spring, ref_line, deflection_line,
164         label
165
166 artists_p = create_car_artists(ax_pass, 'red')
167 artists_a = create_car_artists(ax_act, 'blue')
168 SKIP = 25
169
170 def update_main(frame_idx):
171     idx = frame_idx * SKIP
172     if idx >= len(T): idx = len(T) - 1
173
174     line_pass.set_data(T[:idx], x1_p[:idx])
175     line_act.set_data(T[:idx], x1_a[:idx])
176     line_road.set_data(T[:idx], W[:idx])
177
178     road_x = np.linspace(-2, 4, 150)
179     road_y = np.interp(T[idx] + road_x/velocity, T, W, left=0,
180                        right=0)
181
182     def update_car(artists, x1, x2):
183         road, wheel, body, spring, ref, line, lbl = artists
184         road.set_data(road_x, road_y)
185         wheel.center = (0, x2[idx] + 0.15)
186         by = x1[idx] + 0.65
187         body.set_xy((-0.4, by))
188         spring.set_data([0, 0], [x2[idx]+0.15, by])
189         ref.set_data([-0.4, 0.4], [by+0.9, by+0.9])
190         lbl.set_position((0, by+1.05))
191         win = max(2, int(1.0/dt))
192         start = max(0, idx-win)
193         seg = x1[start:idx+1]
194         if len(seg) > 1:
195             line.set_data(np.linspace(-0.4, 0.4, len(seg)), by
196                           +0.9 + seg*2.0)
197         else: line.set_data([], [])

```



```

190
191     update_car(artists_p, x1_p, x2_p)
192     update_car(artists_a, x1_a, x2_a)
193     return [line_pass, line_act, line_road] + list(artists_p) +
        list(artists_a)
194
195 def update_rms(frame_idx):
196     idx = frame_idx * SKIP
197     if idx >= len(T): idx = len(T) - 1
198     line_rms_p.set_data(T[:idx], rms_p[:idx])
199     line_rms_a.set_data(T[:idx], rms_a[:idx])
200
201     if idx > 200 and frame_idx % 10 == 0:
202         avg_p = np.mean(rms_p[:idx])
203         avg_a = np.mean(rms_a[:idx])
204         if avg_p > 0:
205             imp = (1 - avg_a/avg_p) * 100
206             text_score.set_text(f"Vertical Vib Reduced: +{int(imp
                )}%")
207             if imp > 50: text_score.set_color('green')
208             elif imp > 0: text_score.set_color('orange')
209             else: text_score.set_color('red')
210
211     return [line_rms_p, line_rms_a, text_score]
212
213 ani1 = animation.FuncAnimation(fig1, update_main, frames=len(T)//
    SKIP, interval=1, blit=True)
214 ani2 = animation.FuncAnimation(fig2, update_rms, frames=len(T)//
    SKIP, interval=1, blit=True)
215
216 plt.show()

```

B LQR Simulink Model attributes

B.1 Full model

```

===== MODEL CALLBACKS =====

--- InitFcn ---
(Empty)

--- StartFcn ---
(Empty)

--- StopFcn ---
(Empty)

--- PreLoadFcn ---
(Empty)

```

```
--- PostLoadFcn ---
(Empty)
```

===== STATE-SPACE BLOCKS =====

```
--- Found Block: Active Plant (DiscreteStateSpace) ---
```

```
A: [4x4 double] (Source: Model WS, Var: Ad_a)
B: [4x1 double] (Source: Model WS, Var: Bd_a)
C: [4x4 double] (Source: Base WS, Var: eye(4))
D: [4x1 double] (Source: Base WS, Var: zeros(4,1))
```

```
--- Found Block: Passive Plant (DiscreteStateSpace) ---
```

```
A: [4x4 double] (Source: Model WS, Var: Ad_p)
B: [4x2 double] (Source: Model WS, Var: Bd_p)
C: [4x4 double] (Source: Base WS, Var: eye(4))
D: [4x2 double] (Source: Base WS, Var: zeros(4,2))
```

===== GAIN BLOCKS =====

```
Block: Gain | Gain: -K_lqr
Block: t_div3 | Gain: 0.333333 (Base WS)
```

===== SUM BLOCKS =====

```
Block: Sum | Inputs: +++
Block: Sum | Inputs: |+-
Block: Sum1 | Inputs: |+-
```

===== FROM WORKSPACE =====

```
No FromWorkspace blocks.
```

===== MODEL WORKSPACE VARIABLES =====

Name	Class	Size
Ad_a	double	4x4
Ad_p	double	4x4
Bd_a	double	4x1
Bd_p	double	4x2
K_lqr	double	1x4
Q	double	4x4
R	double	1x1
b1	double	1x1
b2	double	1x1
dt	double	1x1
k1	double	1x1
k2	double	1x1
m1	double	1x1
m2	double	1x1

===== BASE WORKSPACE VARIABLES =====

Name	Size	Bytes	Class
Attributes			

<code>ans</code>	1x1	129571	struct
<code>blk</code>	1x41	82	char
<code>blocks</code>	13x1	2060	cell
<code>callbacks</code>	1x5	686	cell
<code>cbCode</code>	0x0	0	char
<code>cbName</code>	1x11	22	char
<code>count</code>	1x1	8	double
<code>demuxes</code>	2x1	410	cell
<code>dstH</code>	1x1	8	double
<code>dstHandles</code>	1x1	8	double
<code>dstName</code>	1x42	84	char
<code>fw</code>	0x1	0	cell
<code>gains</code>	2x1	402	cell
<code>i</code>	0x0	0	double
<code>info</code>	1x1	104323	struct
<code>j</code>	1x1	8	double
<code>k</code>	1x1	8	double
<code>lines</code>	54x1	432	double
<code>model</code>	1x12	24	char
<code>modelName</code>	1x27	54	char
<code>modelWS</code>	1x1	8	Simulink.
<code>ModelWorkspace</code>			
<code>muxes</code>	1x1	200	cell
<code>mwVars</code>	14x1	19998	struct
<code>srcHandle</code>	1x1	8	double
<code>srcName</code>	1x37	74	char
<code>ssBlocks</code>	2x1	402	cell
<code>ssblk</code>	1x24	48	char
<code>sums</code>	3x1	578	cell
<code>szStr</code>	1x3	6	char
<code>type</code>	1x18	36	char

B.2 Unit-step response model attributes

```

===== MODEL CALLBACKS =====

--- InitFcn ---
m1 = 250; m2 = 45;
k1 = 16000; k2 = 160000;
b1 = 1000; b2 = 0;
dt = 0.001; T_end = 20; t = 0:dt:T_end;

use_sine = false;
if use_sine
    freq = 1.27;
    w = 0.06 * sin(2*pi*freq*t);
    inputType = 'sine';
else
    w = zeros(size(t));

```

```

        w(t > 0.5) = 0.1;
        inputType = 'step';
end

if strcmp(inputType, 'step')
    q_body_vel = 1000;
    q_integral = 5e7;
else
    q_body_vel = 1e8;
    q_integral = 1e8;
end
q_body_pos = 250;
R = 4e-6;

A = [0, 1, 0, 0; -k1/m1, -b1/m1, k1/m1, b1/m1; 0, 0, 0, 1; k1/m2,
      b1/m2, -(k1+k2)/m2, -(b1+b2)/m2];
B_sim = [0, 0; 1/m1, 0; 0, 0; -1/m2, k2/m2];

C_track = [1 0 0 0];
D = [0, 0];

Aa = [A, zeros(4,1); -C_track, 0];
Ba = [B_sim(:,1); 0];
Qa = diag([q_body_pos, q_body_vel, 100, 100, q_integral]);

K_aug = lqr(Aa, Ba, Qa, R);
K_x = K_aug(1:4);
K_i = K_aug(5);

RoadData = timeseries(w, t);
fprintf('Detected Input: %s | Q_vel: %e | Updated K_i: %f\n',
        inputType, q_body_vel, K_i);

--- StartFcn ---
(Empty)

--- StopFcn ---
(Empty)

--- PreLoadFcn ---
(Empty)

--- PostLoadFcn ---
(Empty)

===== STATE-SPACE BLOCKS =====

```

```

--- Found Block: State-Space (StateSpace) ---
A: A
B: B_sim
C: [4x4 double] (Source: Base WS, Var: eye(4))
D: [4x2 double] (Source: Base WS, Var: zeros(4,2))

===== GAIN BLOCKS =====
Block: Gain1          | Gain: K_i
Block: Gain2          | Gain: K_x

===== SUM BLOCKS =====
Block: Sum            | Inputs: |+-
Block: Sum1           | Inputs: |--
Block: Sum2           | Inputs: |+-

===== FROM WORKSPACE =====
Block: From
Workspace             | VariableName: RoadData

===== MODEL WORKSPACE VARIABLES =====
Model Workspace is empty.

===== BASE WORKSPACE VARIABLES =====

```

Name	Size	Bytes	Class
Attributes			
CD	1x16	128	double
CD_8	1x16	128	double
CL	1x16	128	double
CL_8	1x16	128	double
M_inf	1x1	8	double
alpha_deg	1x16	128	double
alpha_rad	1x16	128	double
beta	1x1	8	double
blk	1x24	48	char
c_flap	1x1	8	double
c_main	1x1	8	double
callbacks	1x5	686	cell
cbCode	0x0	0	char
cbName	1x11	22	char
coeffs_factor	1x1	8	double
colors	1x4	488	cell
delta_deg_list	1x4	32	double
delta_rad	1x1	8	double
delta_rad_8	1x1	8	double
delta_val	1x1	8	double
fw	1x1	174	cell
gains	2x1	312	cell
i	1x1	8	double
legends	1x4	608	cell

markers	1x4	488	cell
modelName	1x12	24	char
modelWS	1x1	8	Simulink.
ModelWorkspace			
mwVars	0x1	142	struct
out	1x1	488017	Simulink.
SimulationOutput			
ssBlocks	1x1	168	cell
sums	3x1	460	cell
table_res	16x3	1857	table
type	1x10	20	char